

ИЕРАРХИЯ ПАМЯТИ

Архитектура процессоров и технология CUDA

Перцев Дмитрий – 18.09.2026

Содержание лекции



Часть I. Введение в иерархию памяти

1. Введение. Время доступа к памяти

Часть II. Виртуальная память

1. Виртуальная память. MMU
2. TLB. Проблемы и решения

Часть III. Кэш-память

1. Кэш. Принципы и разновидности
2. Оптимизация доступа к кэшу
3. Когерентность кэша. Протоколы MSI/MESI/MOSI

Часть IV. Регистровый файл



1. Регистровый файл. Типы и реализации

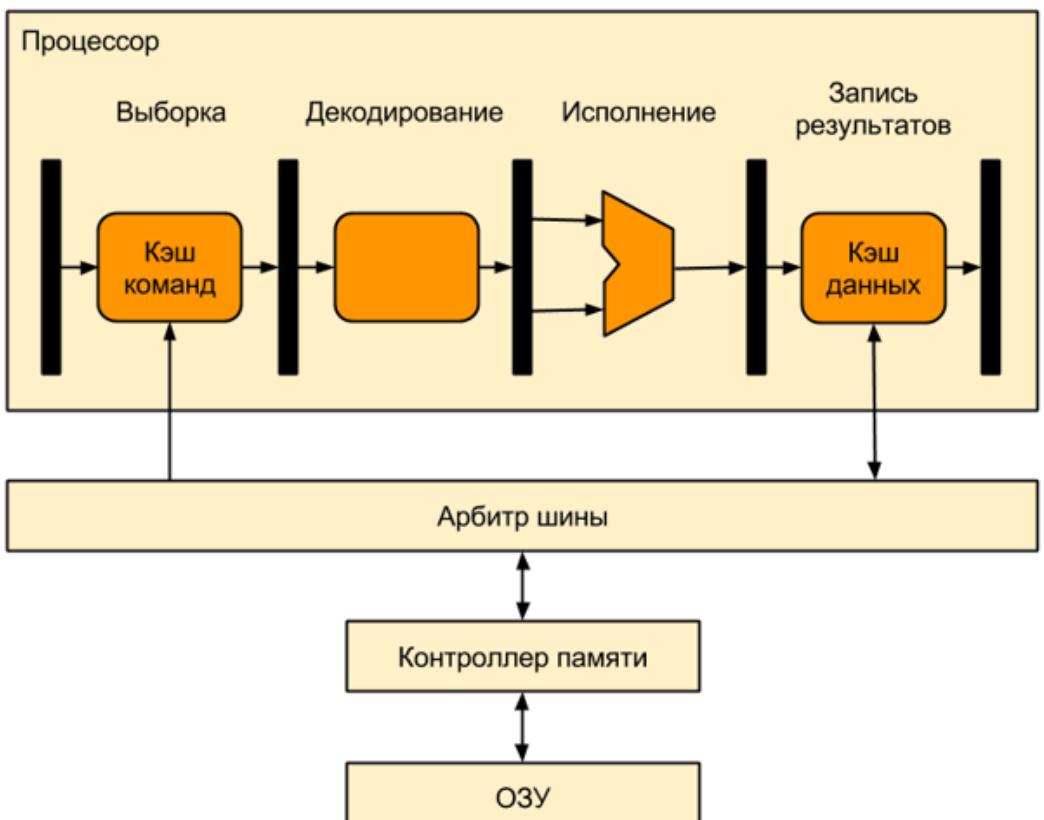


ВВЕДЕНИЕ В ИЕРАРХИЮ ПАМЯТИ

Время доступа к памяти



Структура процессора

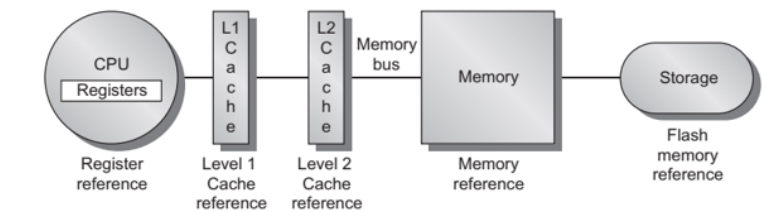


Память

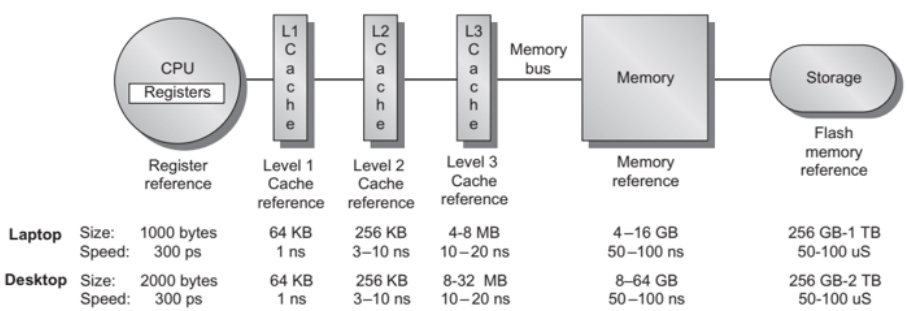


- принцип локальности данных и инструкций
- принцип иерархической организации памяти
  - все адресное пространство памяти доступно в самой большой и самой медленной памяти
  - временная и пространственная локальность гарантирует, что почти все ссылки могут быть найдены в меньшей памяти
    - если текущее обращение к памяти произошло по адресу **A**, следующее обращение произойдет по соседнему адресу
    - если произошло обращение к памяти по адресу **A**, вероятно, что в ближайшее время произойдет еще одно обращение по этому адресу

Иерархия памяти



Memory hierarchy for a personal mobile device



- Hennessy, Patterson (2017)

Проблема доступа к памяти



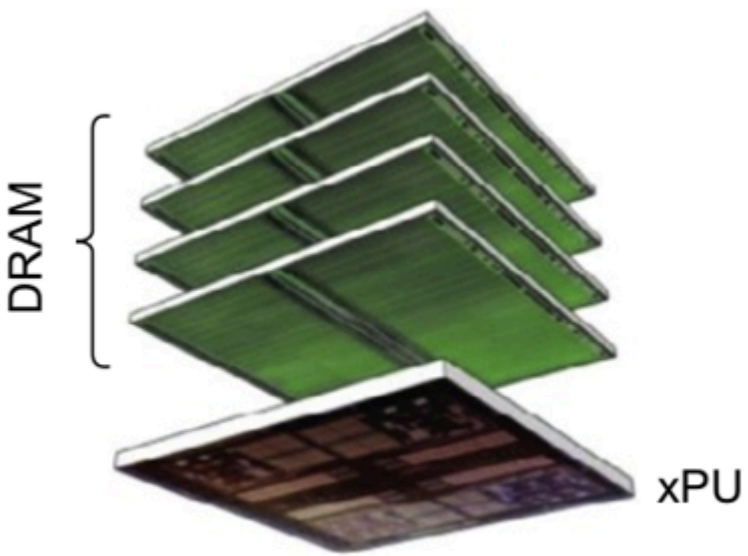
- Intel Core i7 6700 (4x 4,2 GHz)
  - 2 запроса к данным в память за такт от одного ядра
  - 192-битная шина
    - 64-битная шина данных
    - 128-битная шина инструкций
  - чипсет
    - 32,8 млрд. 64-битных данных + 12,8 млрд. 128-битных инструкций в секунду = 409,6 ГВт/с
  - DRAM DDR4: 3200 МГц x 64 бита (размер данных, который может быть передан за 1 такт работы памяти) x 2 (режим работы) = 50,0 ГВт/с
- Hennessy, Patterson (2017)

Сравнение доступа к памяти

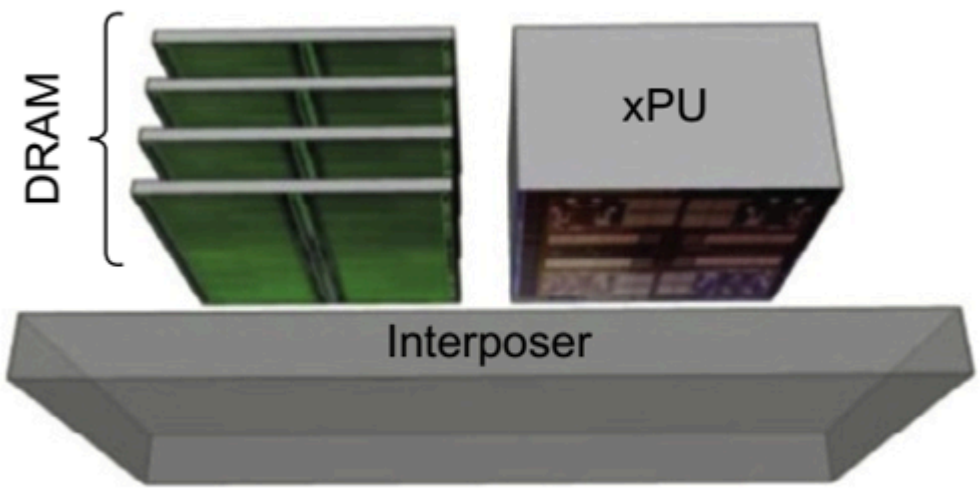


- SRAM
- DRAM DDR4
- GDDR6
- HBM1/2
- Phase-Change Mem (XPoint)

3D XPoint vs HBM



Vertical stacking (3D)



Interposer stacking (2.5D)



ВИРТУАЛЬНАЯ ПАМЯТЬ

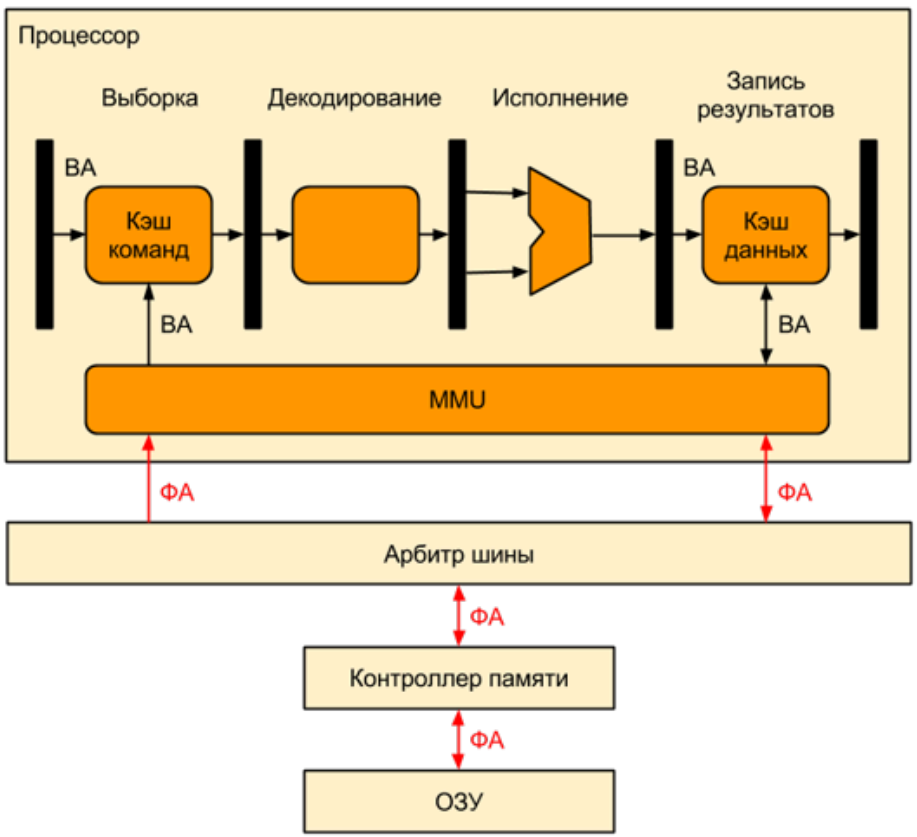
Виртуальная память



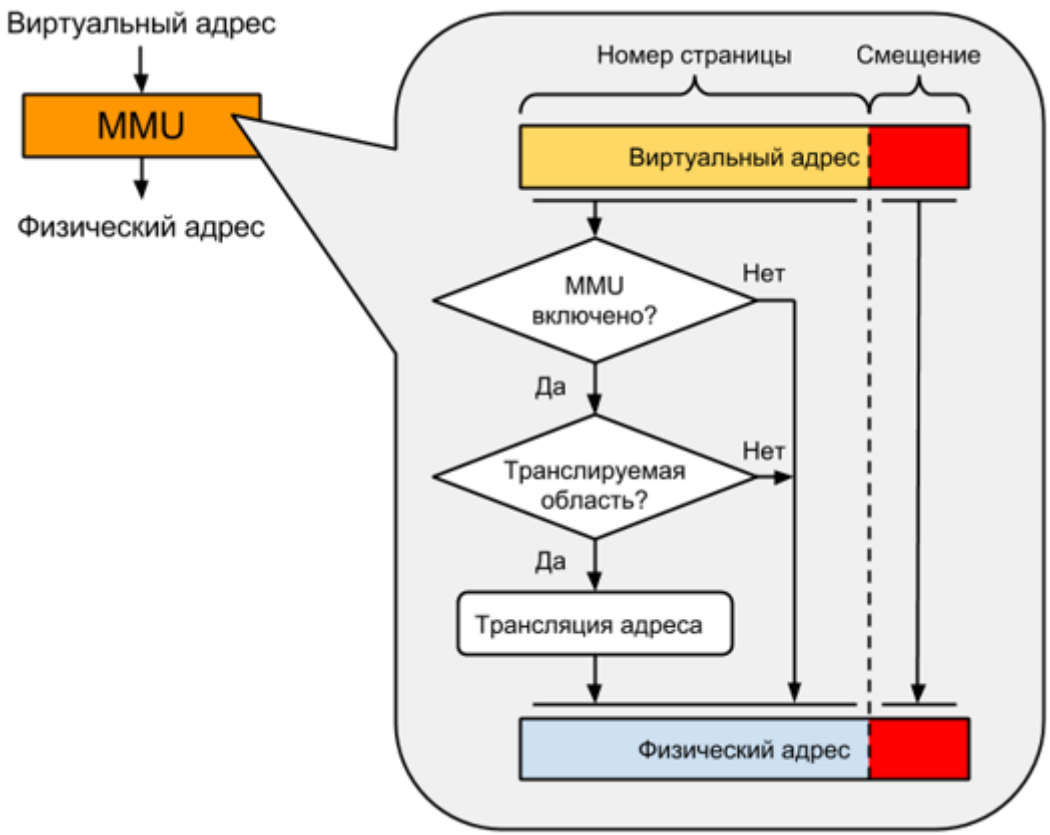
- концепция, которая позволяет уйти от использования физических адресов, использовать вместо них виртуальные
- преимущества:
  - расширение реального адресного пространства
  - согласование изолированных адресных пространств для разных процессов
  - задание разных свойств для разных участков памяти

Структура процессора

- MMU - Memory management unit
- Достоинства:
  - использование при необходимости
  - отсутствие влияния на CPU



Структура MMU

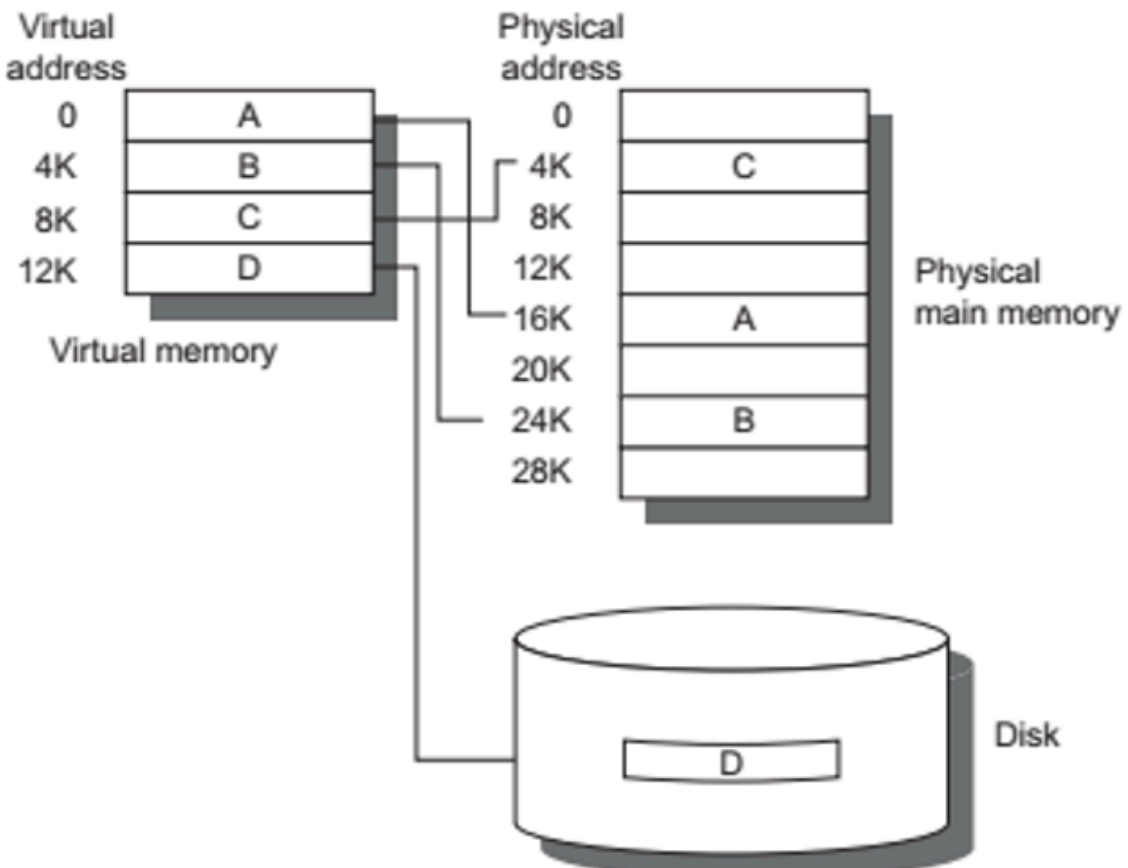


Виртуальная память

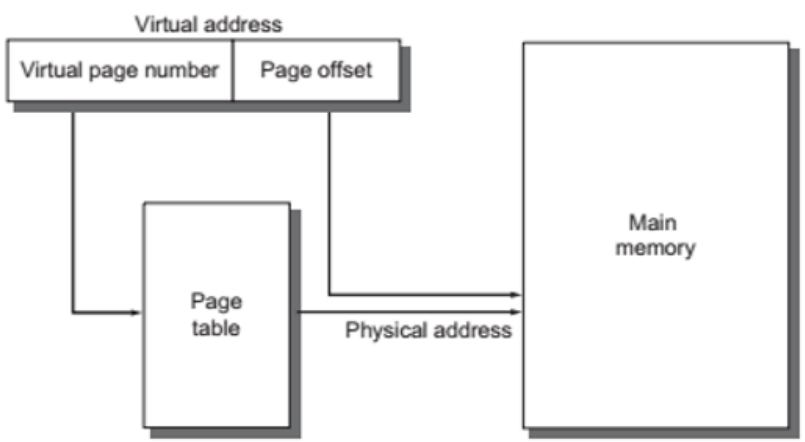


- все физическое адресное пространство процесса разбито на виртуальные страницы
- вся физическая память разбита на физические страницы такого же размера
- у каждого процесса существует таблица, которая связывает виртуальные страницы с физическими страницами
- кроме оперативной памяти страницы могут находиться в файле подкачки

Виртуальная память



Формирование адреса



- Виртуальный адрес разбивается на две части:

1. младшие биты - смещение в странице
2. старшие биты - адрес страницы.

- По таблице преобразования виртуальных адресов виртуальный адрес страницы преобразуется в физический.
- Реальный физический адрес получается конкатенацией физического адреса страницы и смещения.

Описание страницы



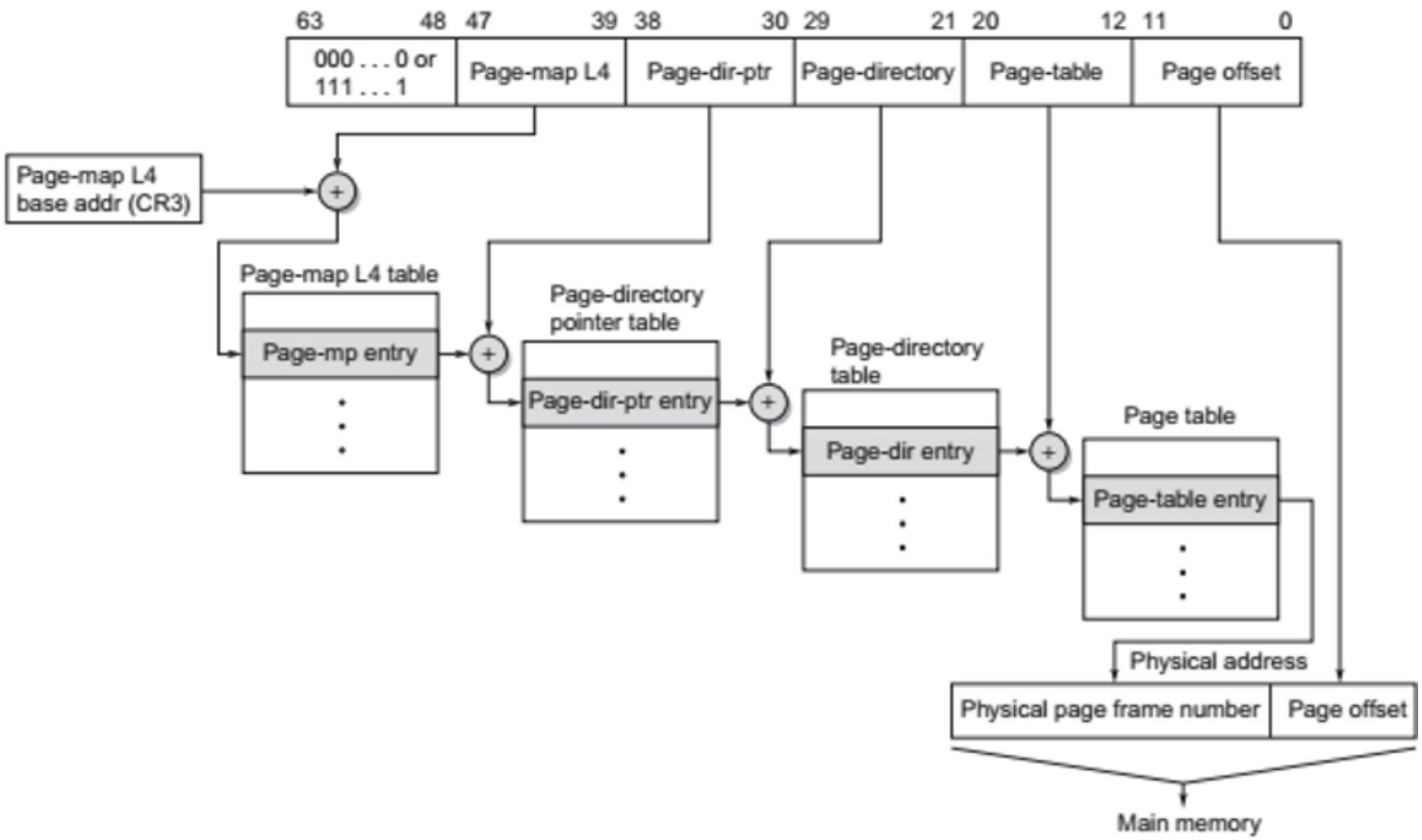
- дескриптор страницы является элементом таблицы преобразования виртуальных адресов
- содержит информацию о
  - физическом адресе страницы
  - текущем расположении (ОЗУ, файл подкачки)
  - атрибутах защиты (можно только читать, исполнять, записывать)
  - изменении страницы
  - выделении страницы физической памяти

Пример таблицы преобразования виртуальных адресов



- Определяется:
  - разрядность адресного пространства
  - размером страницы
  - размером дескриптора страницы
- Пример:
  - Адрес: 32 бита. Размер страницы: 4К. Размер дескриптора 8 байт
  - тогда всего дескрипторов $2^{32-\log_2(4096)} = 1M$, размер таблицы 8М
- Уменьшить количество дескрипторов не возможно, так как для каждой страницы, даже пустой (которой не выделена память), должен быть создан дескриптор

Иерархические таблицы преобразования виртуальных адресов



Сравнение простых и иерархических таблиц преобразования адресов



- простые
  - + преобразование происходит быстро
  - - большой размер таблицы
- иерархические
  - + маленький размер таблицы
  - - большое время преобразование

Виртуальная память, как уровень иерархии памяти



- оперативная память выступает в качестве кэша для «бесконечной» дисковой памяти
- алгоритм вытеснения страниц из памяти LRU
- за перенос страниц между диском и О отвечает ОС
  - в случае отсутствия требуемой страницы в О процессор генерирует исключение
- при вытеснении страниц из памяти только модифицированные страницы сохраняются на диск

Когда нужно преобразовывать адреса?



- данные
  - любые обращения к данным
- инструкции
  - при выборке инструкций
- вывод: процесс преобразования должен происходить максимально быстро

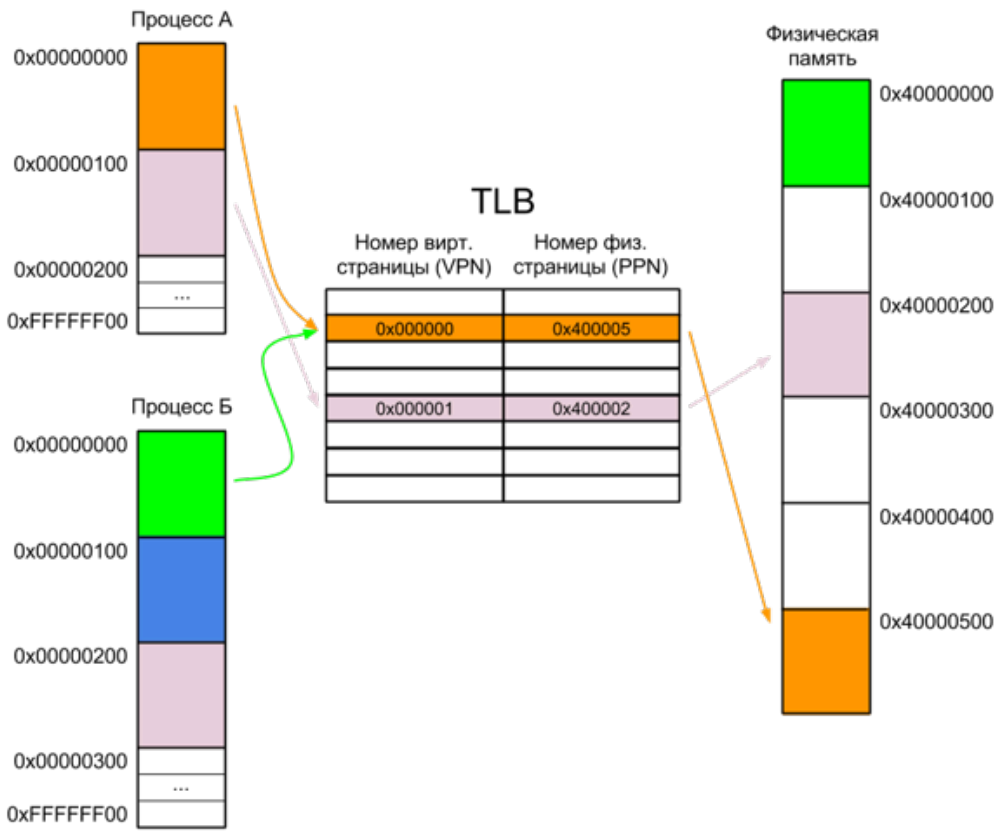
TLB (translation look-aside buffer)
кэш преобразованных адресов



- на вход TLB приходит виртуальный адрес страницы
- Элемент TLB - дескриптор страницы
- процессор содержит два TLB - один для данных, второй для инструкций
- кэш TLB обычно полностью ассоциативный
- может иметь несколько уровней
- при переключении процесса TLB очищается

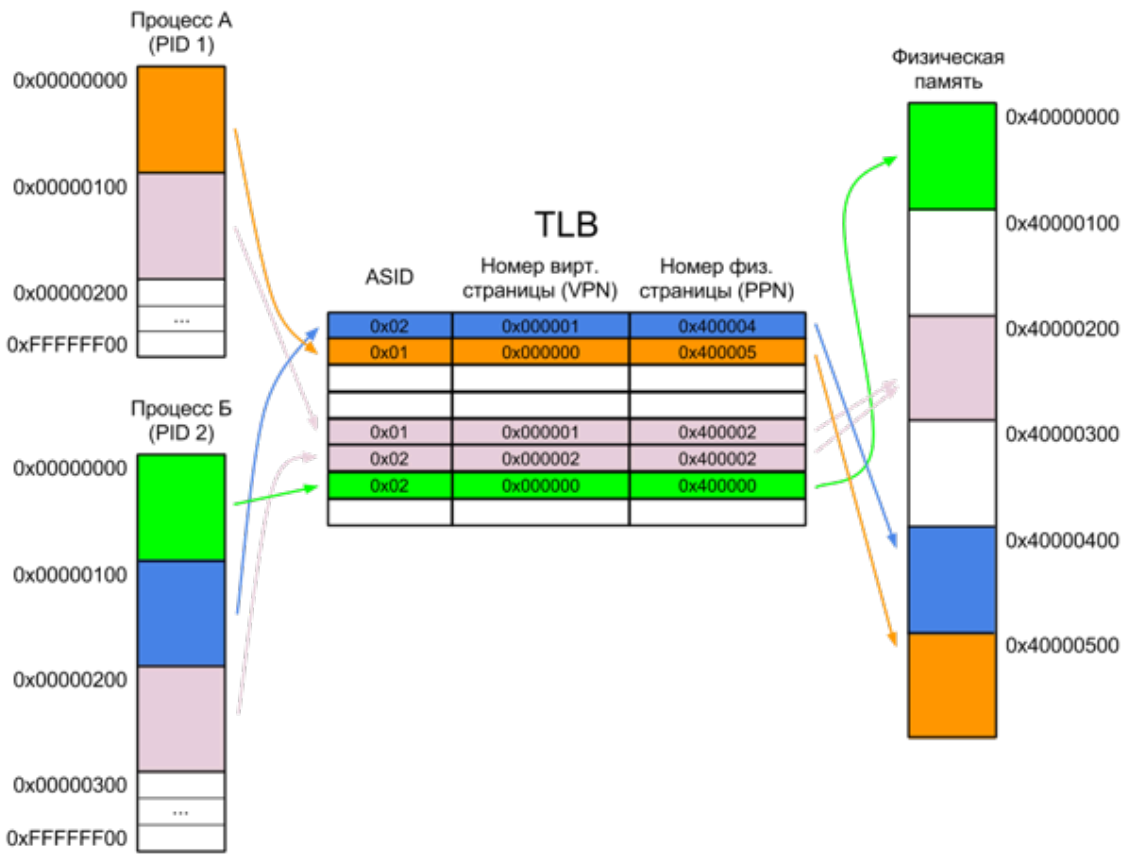
Проблемы TLB

- Размер страницы: 256 байт
- Пример архитектуры:
  - ARM9
- Решение:
  - Отмечать как недействительные
  - Разные виртуальные адресные пространства

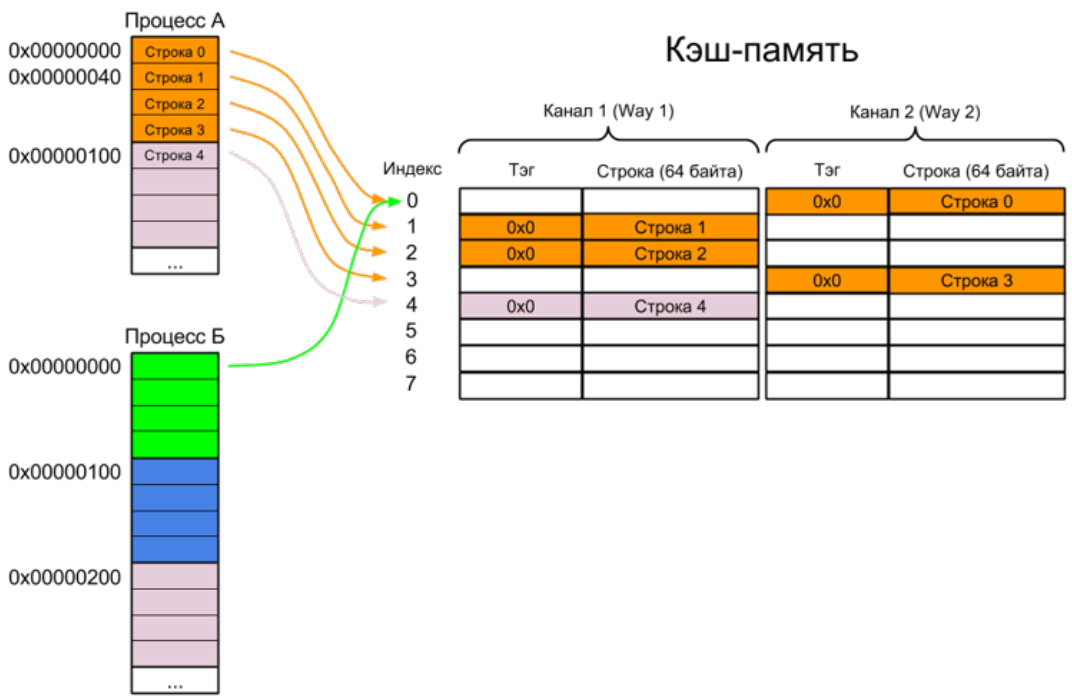


Решение, получившее распространение

- Расширение адреса: 32-40 бита
  - 8 бит PID (исключение ARMv8)
  - 24 бита номер виртуальной страницы



Основная проблема

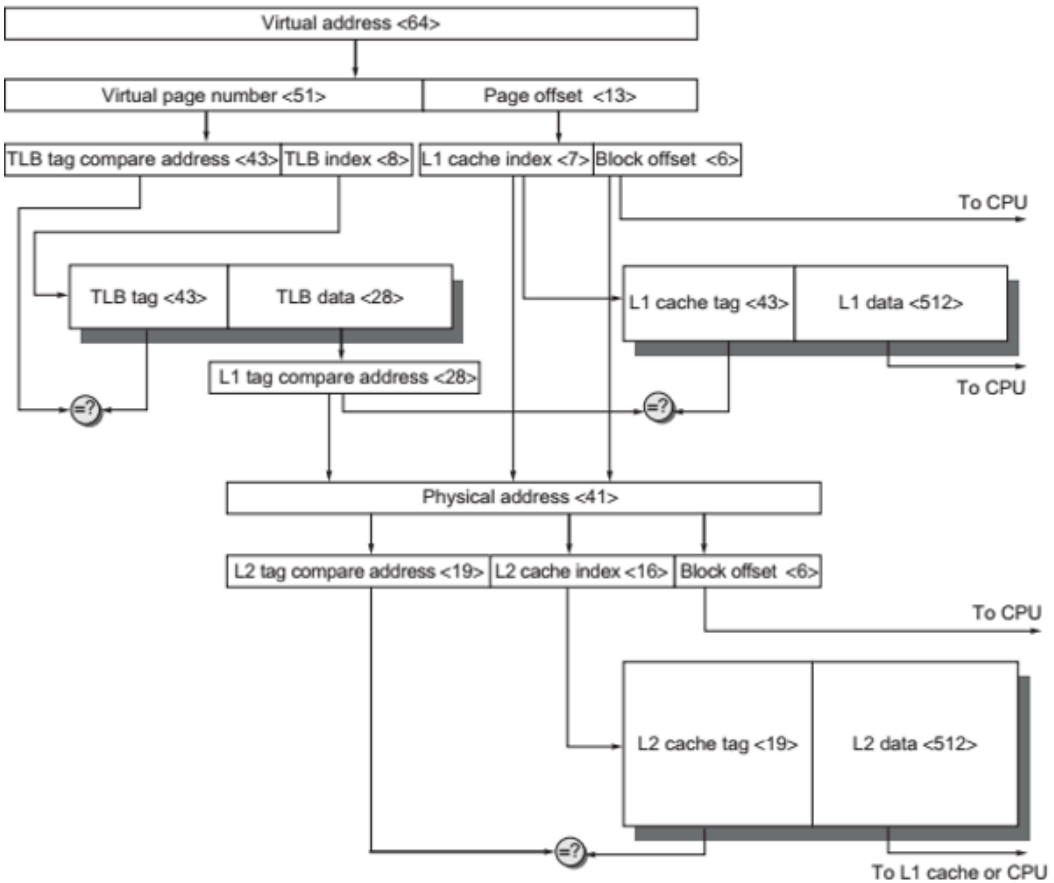


Основная проблема. Решение



- cache flush
- Добавить ASID в TLB кэша
- Обращаться в MMU каждый раз, когда мы обращаемся в кэш (ARM9)

Организация TLB на примере AMD Opteron



Рекомендуемая литература



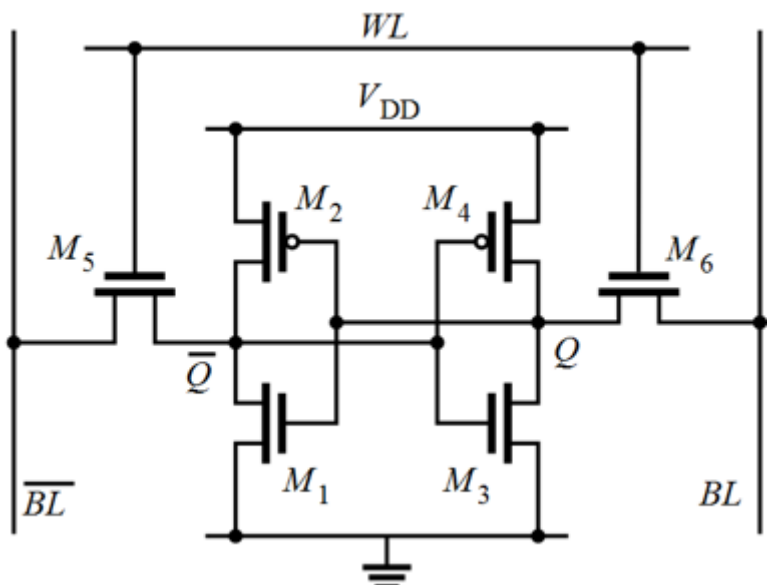
- MMU в картинках (часть 1)
- Организация виртуальной памяти (системное программирование)

кэш



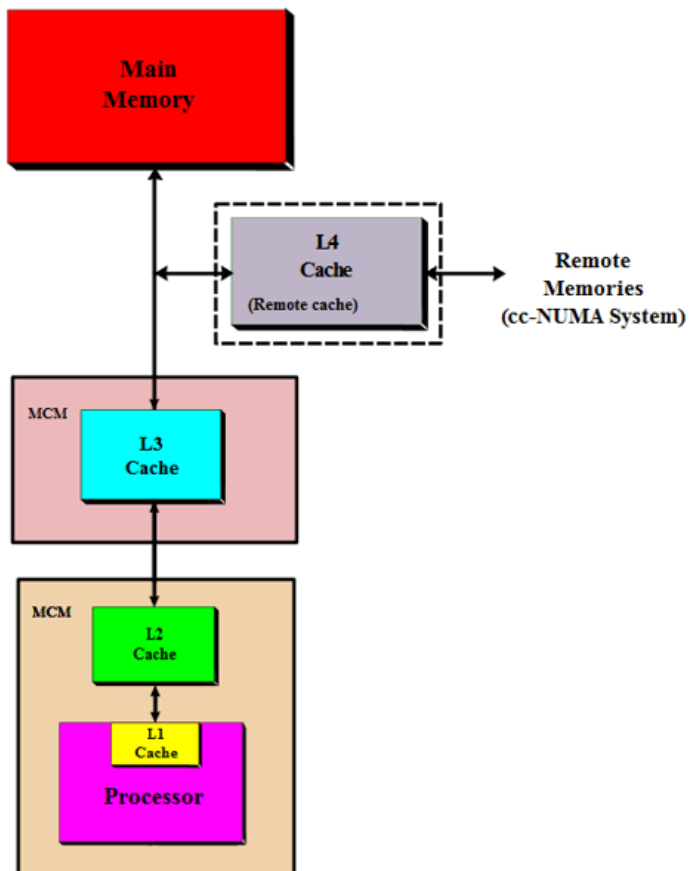
Принципы

- назначение
- свойство локальности данных и инструкций
- на физическом уровне: SRAM

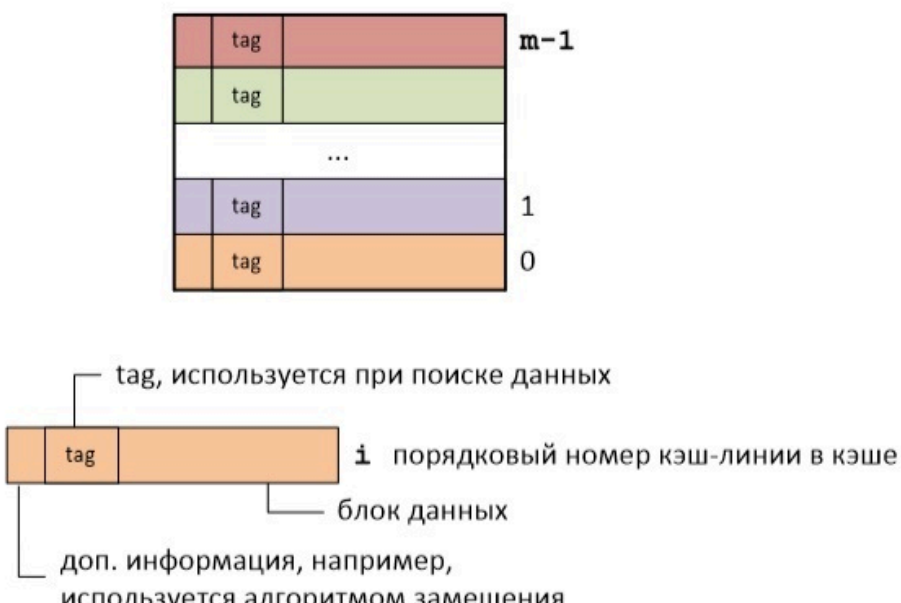


Иерархия кэша

- разновидности кэша:
  - Инклюзивный
    - данные во всех кэшах
  - эксклюзивный
  - write-through
    - обновляет память
  - write-back



Базовая структура кэша



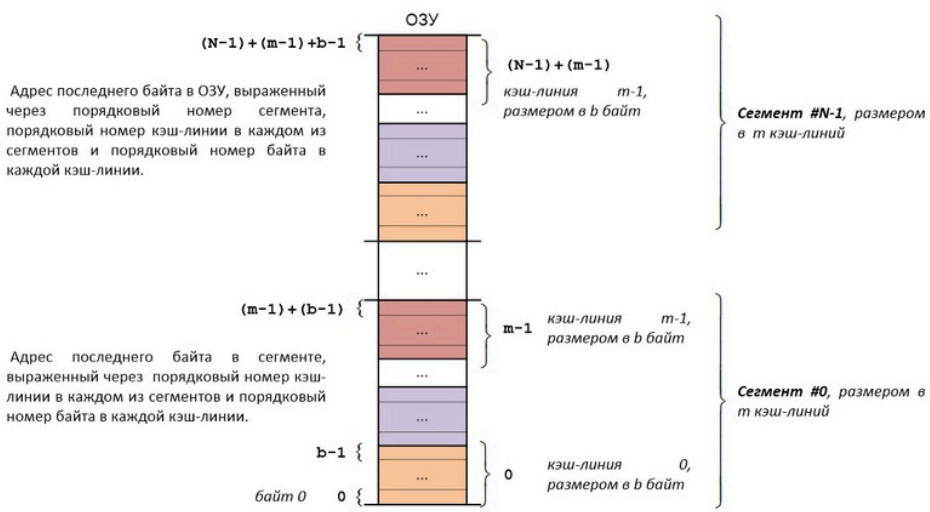
- m кэш линий
- размер кэш линии в x86:
  - 64 байта
  - + служебная информация
- варианты кэш-линии:
  - 4-512 байт

Разновидности кэшей



- с прямым отображением (direct mapping)
- полностью ассоциативная (fully associative mapping)
- set associative mapping

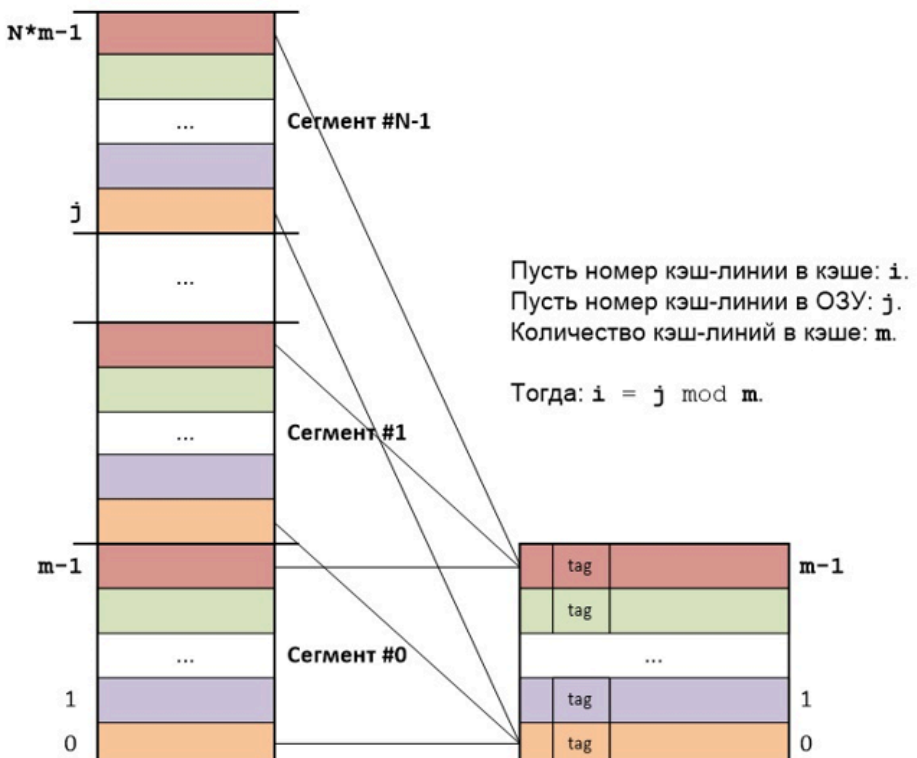
Кэш с прямым отображением



- $sizeof(segment)$ = Размер кэша
- $sizeof(block)$ = кэш-линии

Отображение RAM в кэш

- блок RAM - фиксированная кэш-линия



Адресация кэша

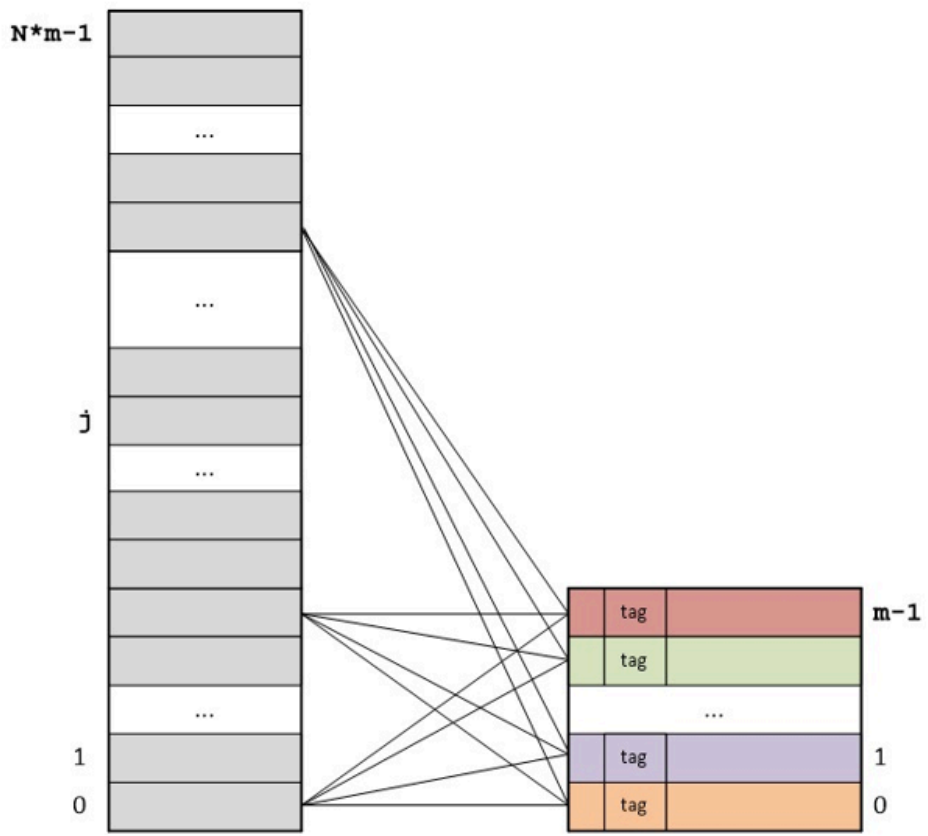


- $address = segment_num + cache_line_num + offset$
  - tag определяет старшую часть адреса первого байта
- для адресации байта потребуется:
  - $\log_2 N + \log_2 m + \log_2 b$ бит
    - N = объем RAM/ размер сегмента = число сегментов
    - m = объем кэш-памяти/ размер кэш линии = количество кэш-линий в кэше
    - b размер кэш-линии
  - определение попадания:
    - извлекается средняя часть адреса ($\log_2 m$), определяющая номер кэш-линии в кэше



- TLB кэш-линии с данным номером сравнивается со старшей частью адреса ($\log_2 N$)

Fully associative mapping

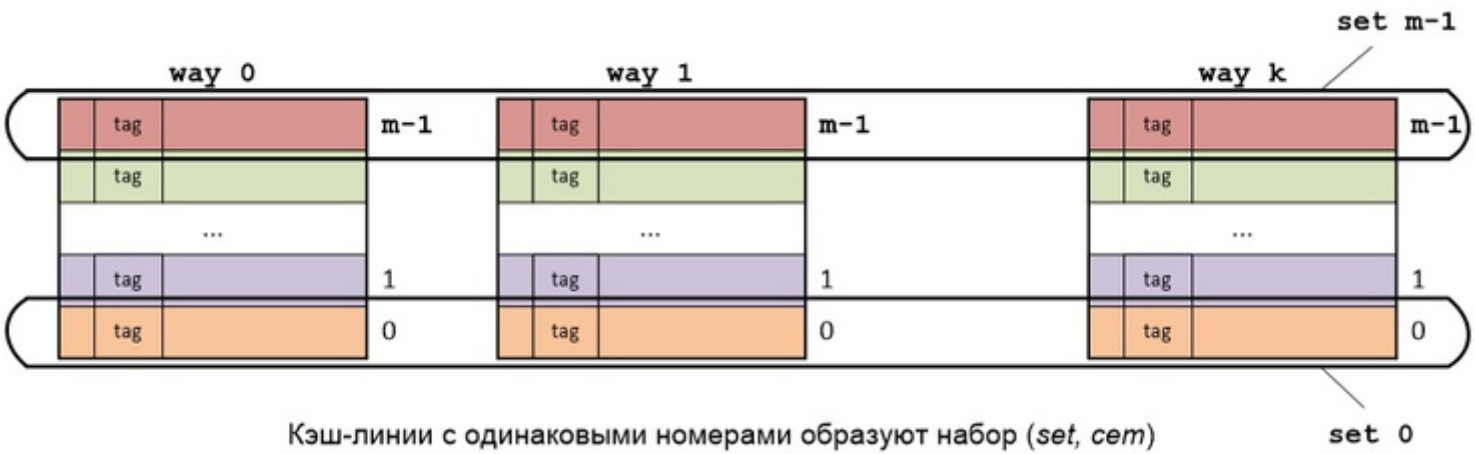


Адресация кэша



- $address = cache_line_num + offset$
- для адресации байта потребуется:
  - $\log_2 m + \log_2 b$ бит
    - m = объем RAM/ размер кэш линии = количество кэш-линий, помещающихся в RAM
    - b размер кэш-линии
  - определение попадания:
    - тэги всех кэш-линий сравниваются со старшей частью адреса одновременно
  - варианты применения:
    - TLB-кэш

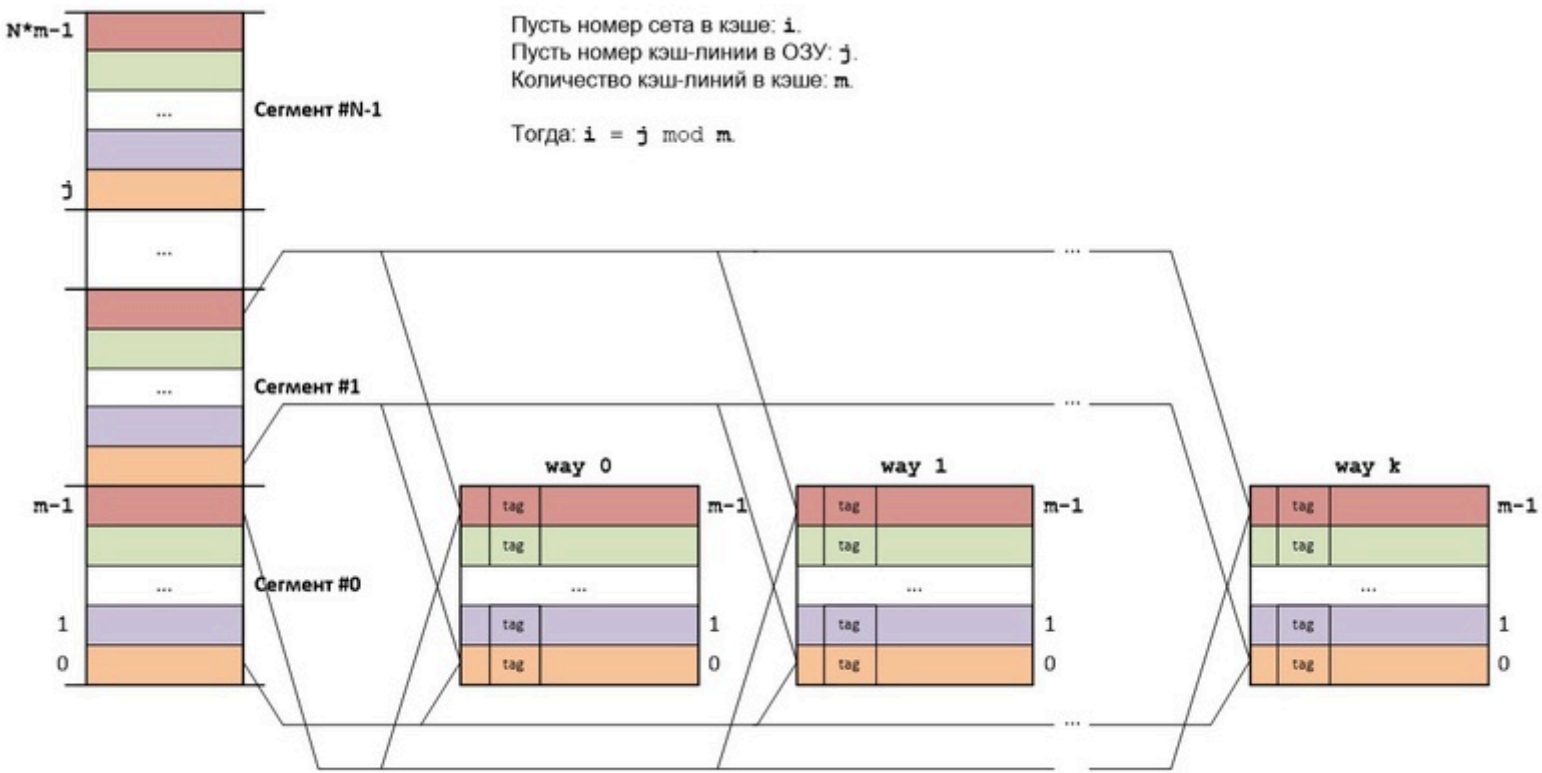
Set associative mapping



Кэш-линии с одинаковыми номерами образуют набор (*set, cem*)

- RAM делится также как и в прямом отображении
- кэш состоит из k кэшей (k каналов), использующих прямое отображение
  - set - кэш-линии, имеющие одинаковые номера во всех каналах
  - значения k: 2-48 (AMD Athlon) / 128 (IBM Power3)

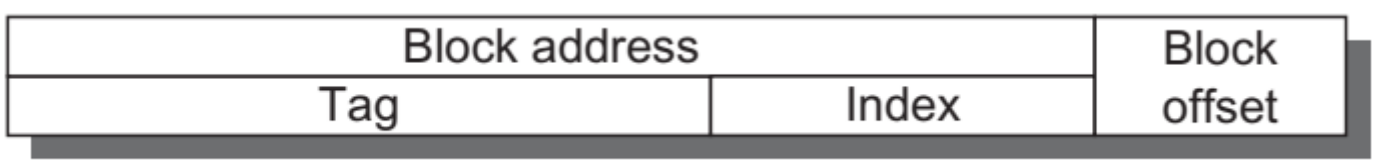
Отображение RAM в кэш





- Блоки RAM из разных сегментов, но с одинаковыми номерами в этих сегментах, всегда будут отображаться на один и тот же set кэша

Структура адреса



Адресация в кэше



- для адресации байта потребуется:
  - $\log_2 N + \log_2 m + \log_2 b$ бит
    - N = объем RAM/ размер сегмента = число сегментов
    - m = объем кэш-памяти/ размер кэш линии = количество кэш-линий в кэше
    - b размер кэш-линии
  - определение попадания:
    - извлекается средняя часть адреса ($\log_2 m$), определяющая номер сета в кэше
    - TLB всех кэш-линий данного сета сравниваются со старшей частью адреса ($\log_2 N$) одновременно

Определение характеристик отображения в CPU

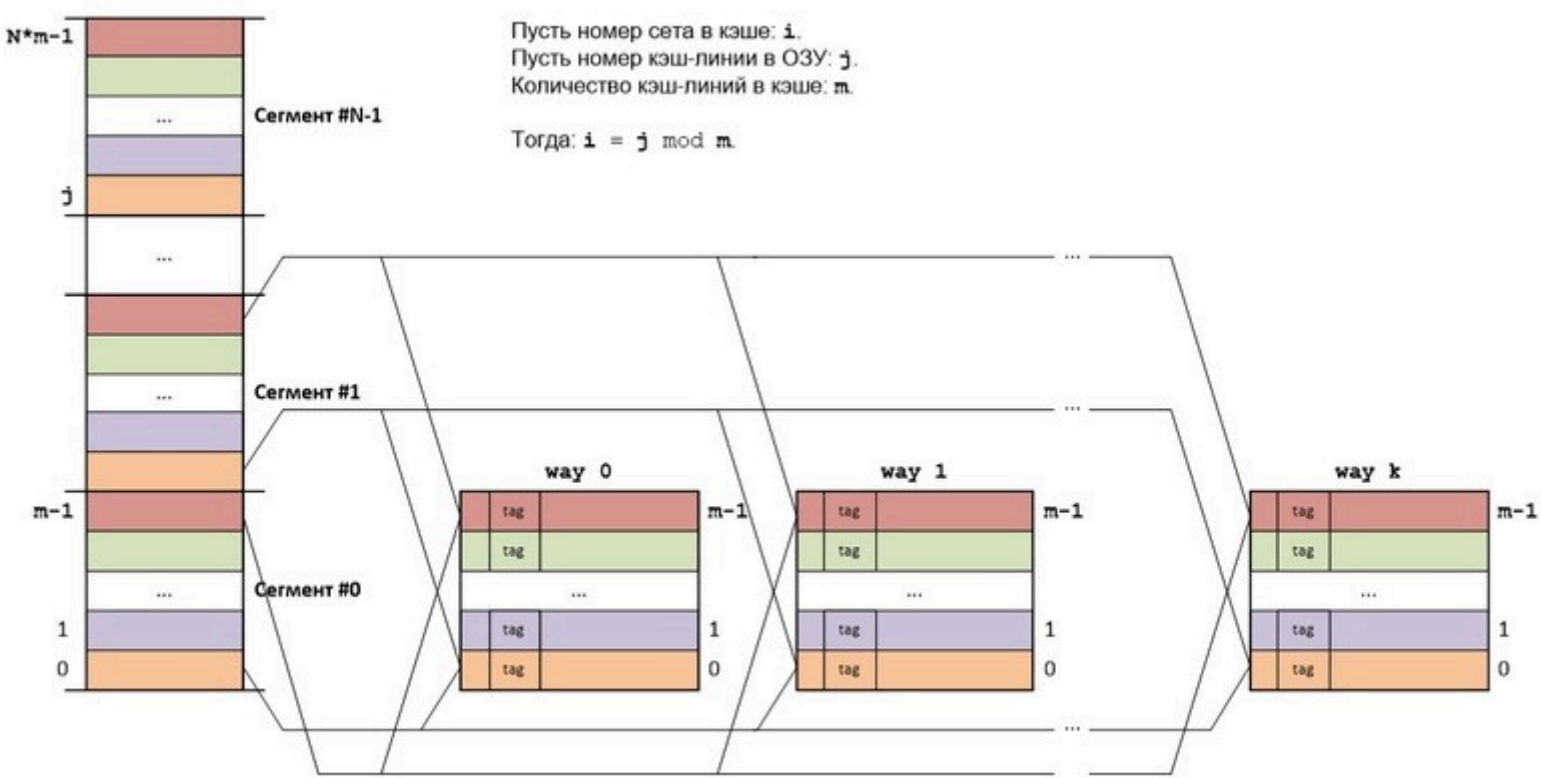


- Linux: `/sys/devices/system/cpu/cpuX/cache/indexY/Z`
  - `X` = интересующее ядро
  - `Y` интересующий кэш (L1 instruction, L1 data, L2, L3)
  - `Z` = что конкретно интересует
- Результат на Intel Xeon X5650:
- `cat /sys/devices/system/cpu/cpu0/cache/index0/size`
  - 32K = размер
- `cat /sys/devices/system/cpu/cpu0/cache/index0/ways_of_associativity`
  - 8 = ассоциативность сета
- `cat /sys/devices/system/cpu/cpu0/cache/index0/number_of_sets`
  - 64 = количество этих сетов

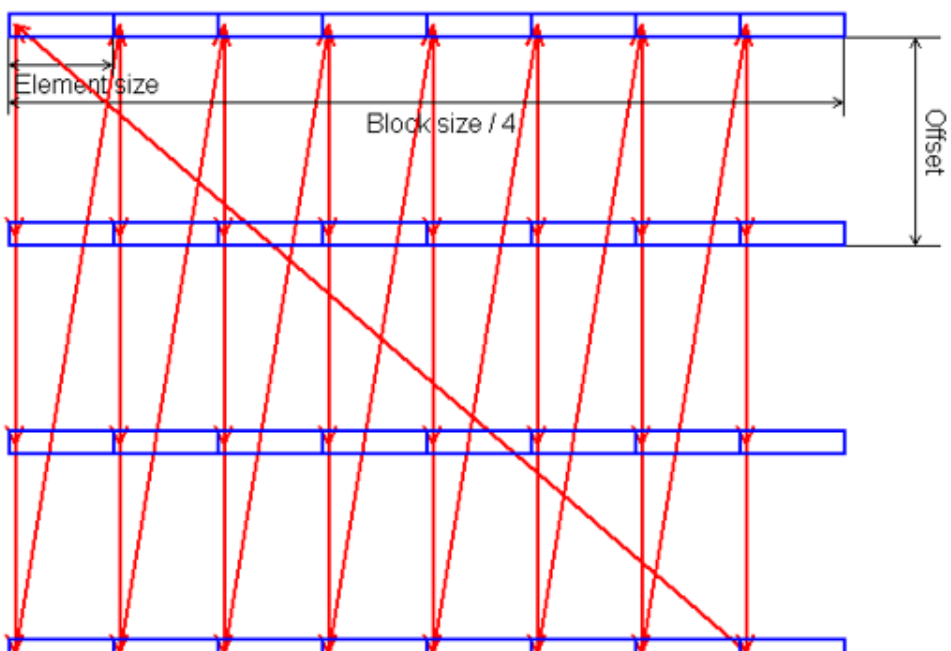


- `cat /sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size`
  - 64 = размер кэш линии

Set associative mapping



Лаб.работа



Среднее время доступа



- Среднее время доступа:

  $[HitTime + MissRate * MissPenalty]$

  $[HitTime_{L1} + MissRate_{L1} * (HitTime_{L2} + MissRate_{L2} * MissPenalty_{L2})]$
- $HitTime$ = время, необходимое на определение попадания в кэш
- $MissRate$ (процент промахов) = отношение числа промахов к общему числу обращений в кэш
- $MissPenalty$ = время, необходимое для замещения кэш-линии из памяти

Пути оптимизации доступа в кэш. Первый случай



- Увеличение размера кэш-линии
  - Уменьшает процент промахов
  - Увеличивает «стоимость» промаха (miss penalty)
  - Возможно увеличение числа замещений в кэше
- Увеличение размера самого кэша
  - Уменьшает процент промахов
  - Увеличение энергопотребления
  - Возможно увеличение hit time
- Увеличение степени ассоциативности
  - Уменьшает процент промахов
  - Возможно увеличение hit time

Пути оптимизации доступа в кэш. Первый случай



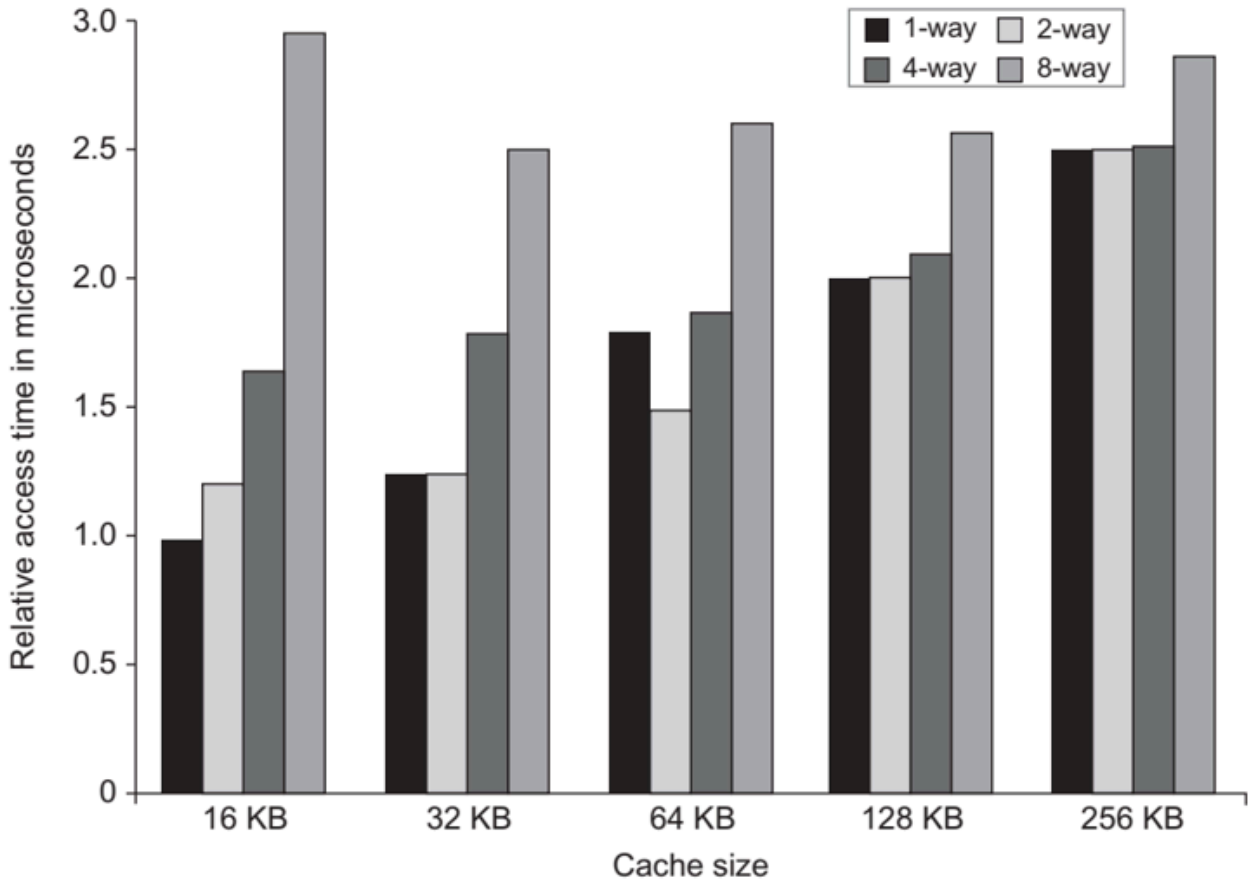
- Многоуровневый кэш
  - Уменьшение стоимости промаха
- Применение отложенной записи (write-back)
  - Приоритет на чтение
  - Записи осуществляются через дополнительный буфер
- Минимизация вычислений
  - виртуальный физический адреса

Оптимизации быстродействия. Практика



- Маленький и максимально управляемый кэш L1
  - Назначение:
    - Максимальное быстродействие
  - Проблема:
    - Затруднено увеличение объема
    - Минимизация hit time
    - Минимизация затрачиваемой энергии
    - Увеличение степени ассоциативности

Оптимизации быстродействия. Практика (CUSTI)



Оптимизации быстродействия. Практика



- Предсказание переходов
  - Минимизация hit time
- Конвейеризация доступа и многобанковый доступ
  - Повышение пропускной способности
  - Чаще применяется для кэша инструкций
  - Пример конвейеризации:
    - Intel Pentium (середина 1990-х) = 1 clock cycle;
    - Pentium Pro - III (начало 2000-х) = 2 clock cycles;
    - с Pentium 4 (2000 г.) = 4 clock cycles

Оптимизации быстродействия. Практика



- Неблокирующий кэш
  - Применяется для L2/L3
  - Пример: выборка инструкций при ожидании получения данных из памяти
- Использование write-back кэша
- Запрашиваемое слово первым и раннее возобновление выполнения для уменьшения потерь на промахи
  - не дожидаться загрузки всей кэш-линии, передавать данные по мере попадания в кэш
- Программная и аппаратная предвыборка данных
  - PREFETCH / PREFETCHA
- Компиляторные оптимизации для уменьшения доли промахов

Оптимизации быстродействия. Практика



- Перестановка циклов
- Разбиение на блоки

```
for (j = 0; j < 100; j = j + 1)
    for (i = 0; i < 5000; i = i + 1)
        x[i][j] = 2 * x[i][j];
```

Оптимизации быстродействия. Практика



- Оптимизация циклов при компиляции

```
// Before
for (j = 0; j < 100; j = j + 1)
    for (i = 0; i < 5000; i = i + 1)
        x[i][j] = 2 * x[i][j];

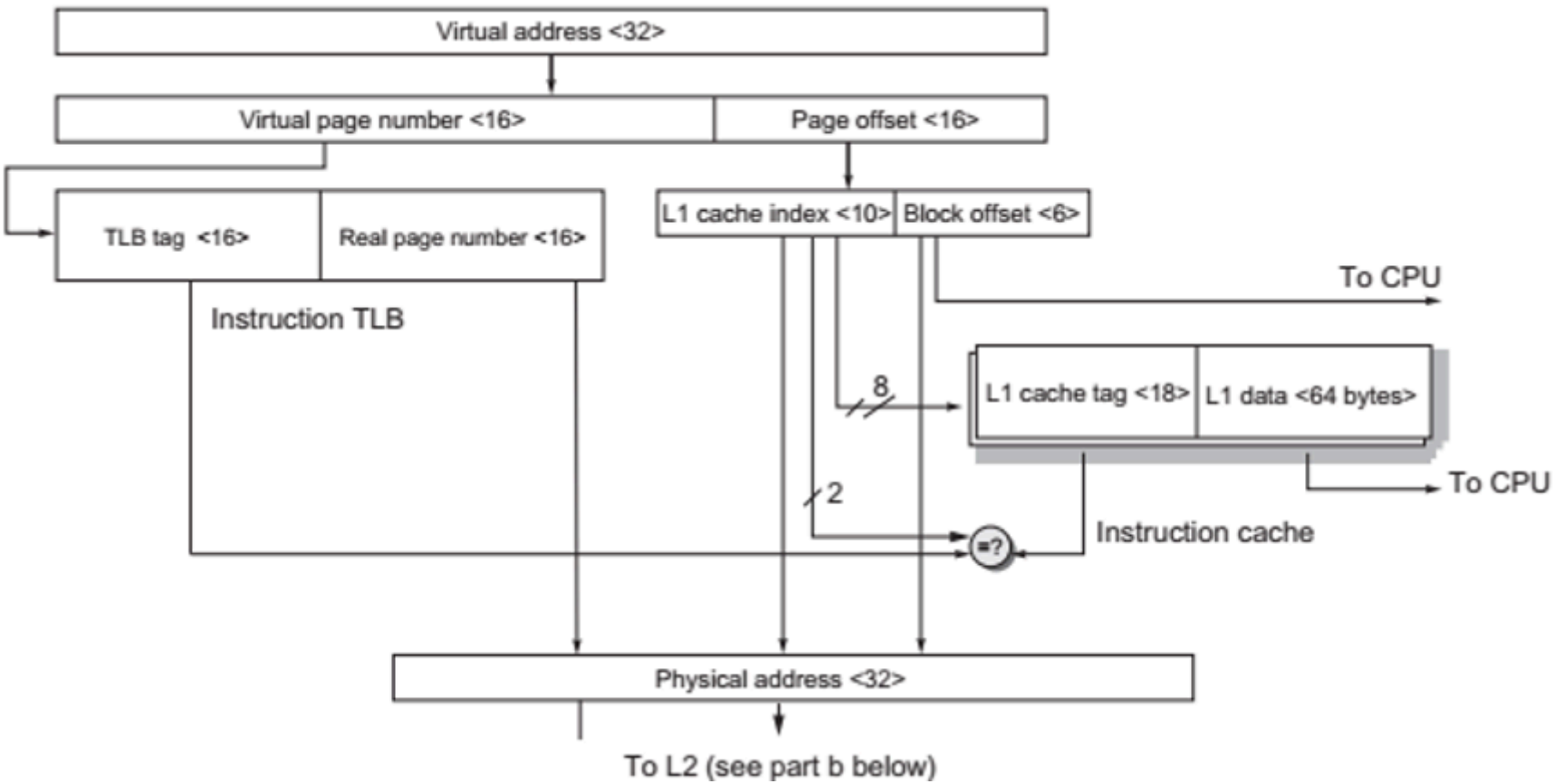
// After
for (i = 0; i < 5000; i = i + 1)
    for (j = 0; j < 100; j = j + 1)
        x[i][j] = 2 * x[i][j];
```

Пример организации памяти в Cortex-A53

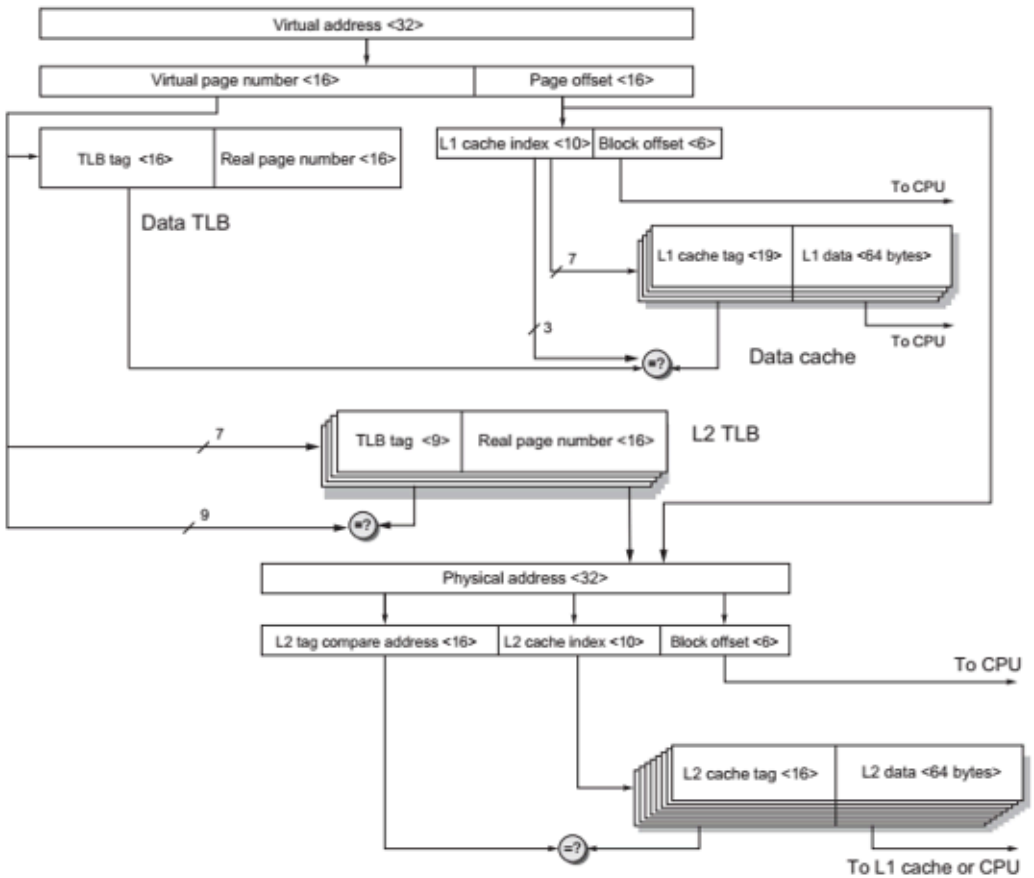


Structure	Size	Organization	Typical miss penalty (clock cycles)
Instruction MicroTLB	10 entries	Fully associative	2
Data MicroTLB	10 entries	Fully associative	2
L2 Unified TLB	512 entries	4-way set associative	20
L1 Instruction cache	8–64 KiB	2-way set associative; 64-byte block	13
L1 Data cache	8–64 KiB	2-way set associative; 64-byte block	13
L2 Unified cache	128 KiB to 2 MiB	16-way set associative; LRU	124

Пример организации памяти в Cortex-A53. Кэш инструкций



Пример организации памяти в Cortex-A53. Кэш данных



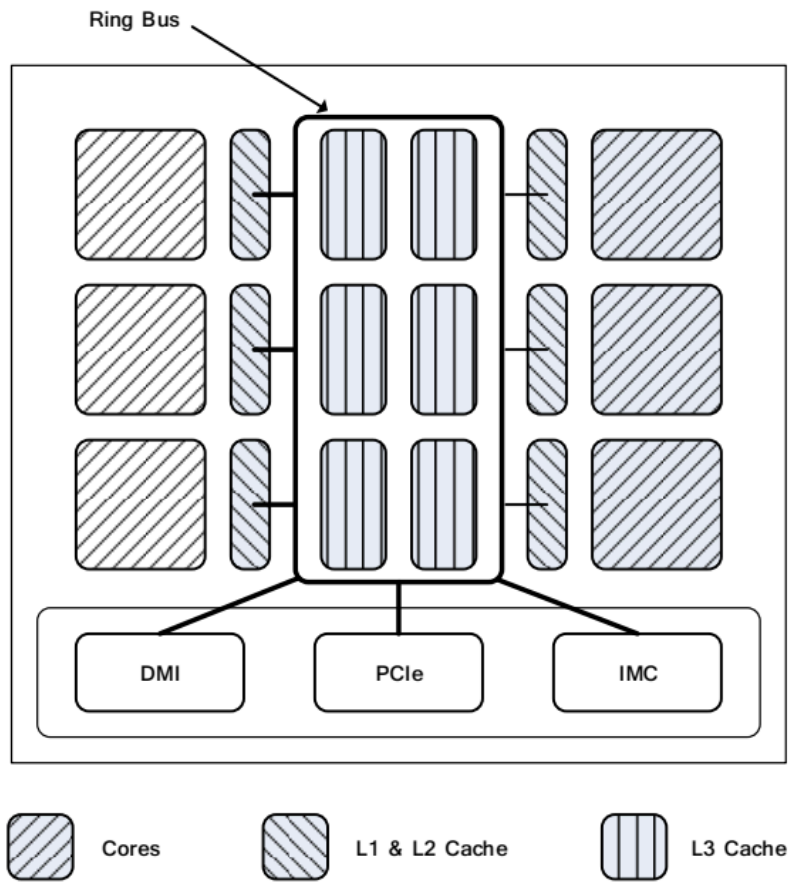
Пример организации памяти в Intel Core i7 6700



Characteristic	Instruction TLB	Data DLB	Second-level TLB
Entries	128	64	1536
Associativity	8-way	4-way	12-way
Replacement	Pseudo-LRU	Pseudo-LRU	Pseudo-LRU
Access latency	1 cycle	1 cycle	8 cycles
Miss	9 cycles	9 cycles	Hundreds of cycles to access page table

Characteristic	L1	L2	L3
Size	32 KiB I/32 KiB D	256 KiB	2 MiB per core
Associativity	both 8-way	4-way	16-way
Access latency	4 cycles, pipelined	12 cycles	44 cycles
Replacement scheme	Pseudo-LRU	Pseudo-LRU	Pseudo-LRU but with an ordered selection algorithm

Сценарий №2: slice



Определения



- C_{cache} объем кэша
- C_{cache} объем памяти
- $C_{cachelin}$ размер кэш-линии
- $a = associativity$ степень ассоциативности
- Общее число блоков:

$$n_{block} = \frac{C_{memory}}{C_{cacheline}}$$

- Общее число set'ов:

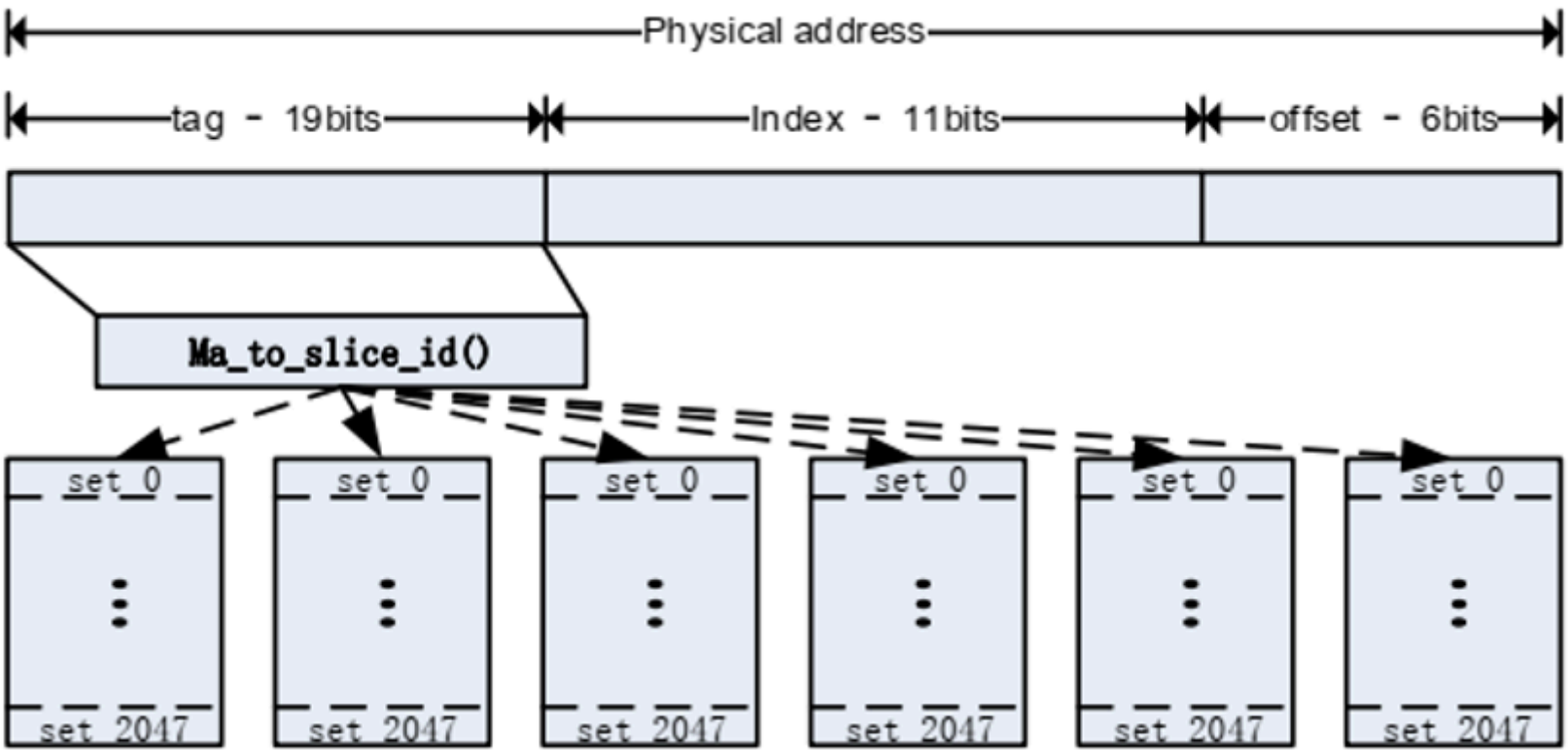
$$n_{set} = \frac{C_{cache}}{a \times C_{cacheline}}$$

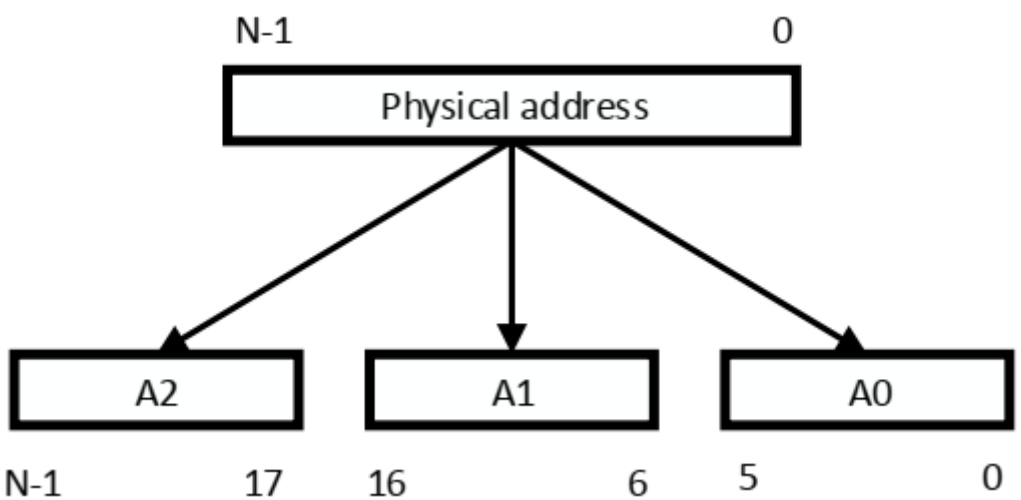
Определения



- Местоположение данных в кэше:
  - $slice_id = ma_to_slice_id(pa)$
  - $set_index = ma_to_set_index(pa)$
    - pa физический адрес

Физическая модель организации Intel Sandy Bridge LLC





- the value of A0 is used as block address;
- the value of A1 is used as set index on a specified cache slice;
- A2 is used to decide the slice id of a given pa
- ...



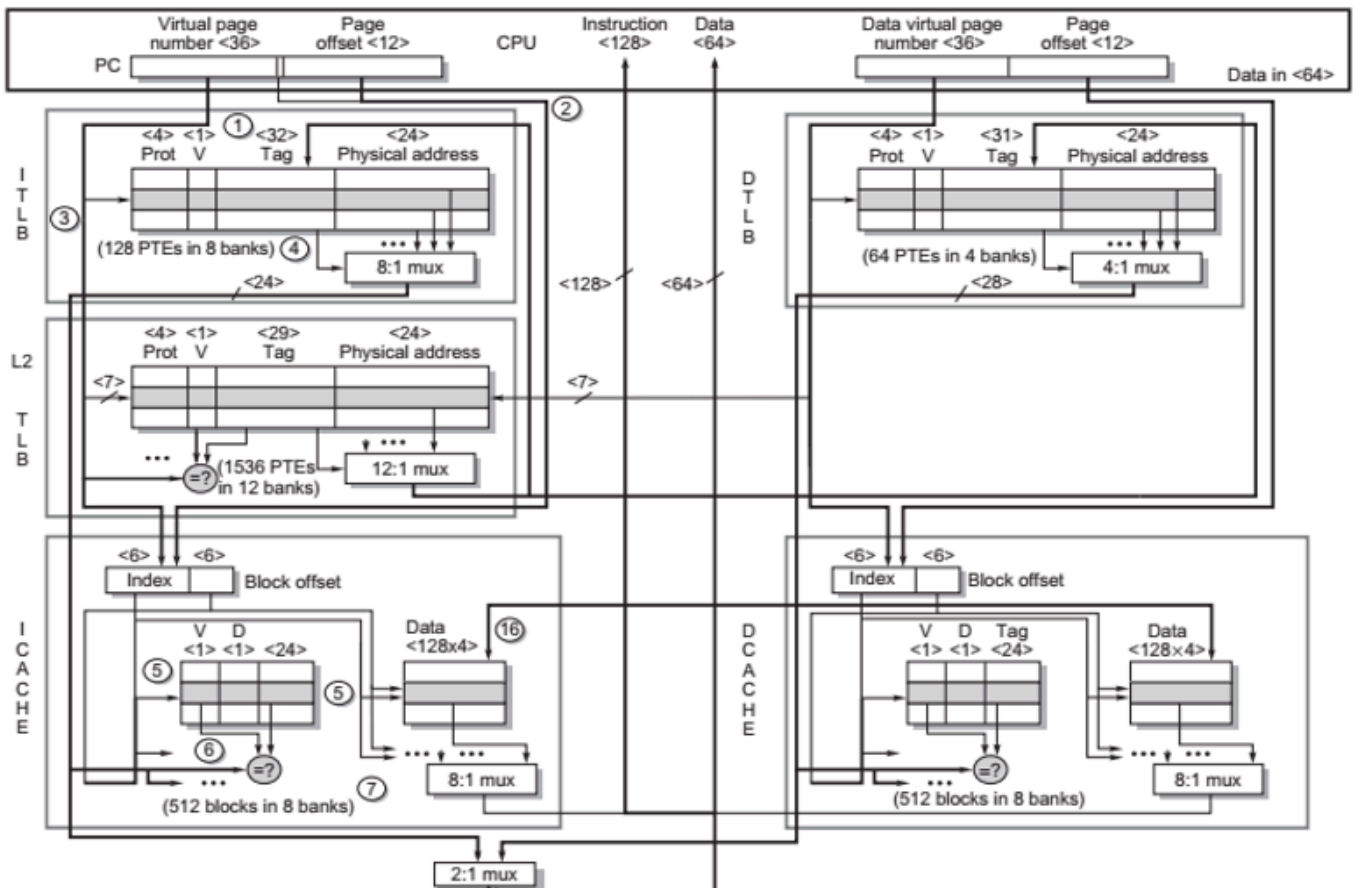
- общее количество блоков данных равно

$$n_{\{block\}} = \frac{C_{memory}}{C_{capacity}} = \frac{16GB}{64B} = 2^{28}$$

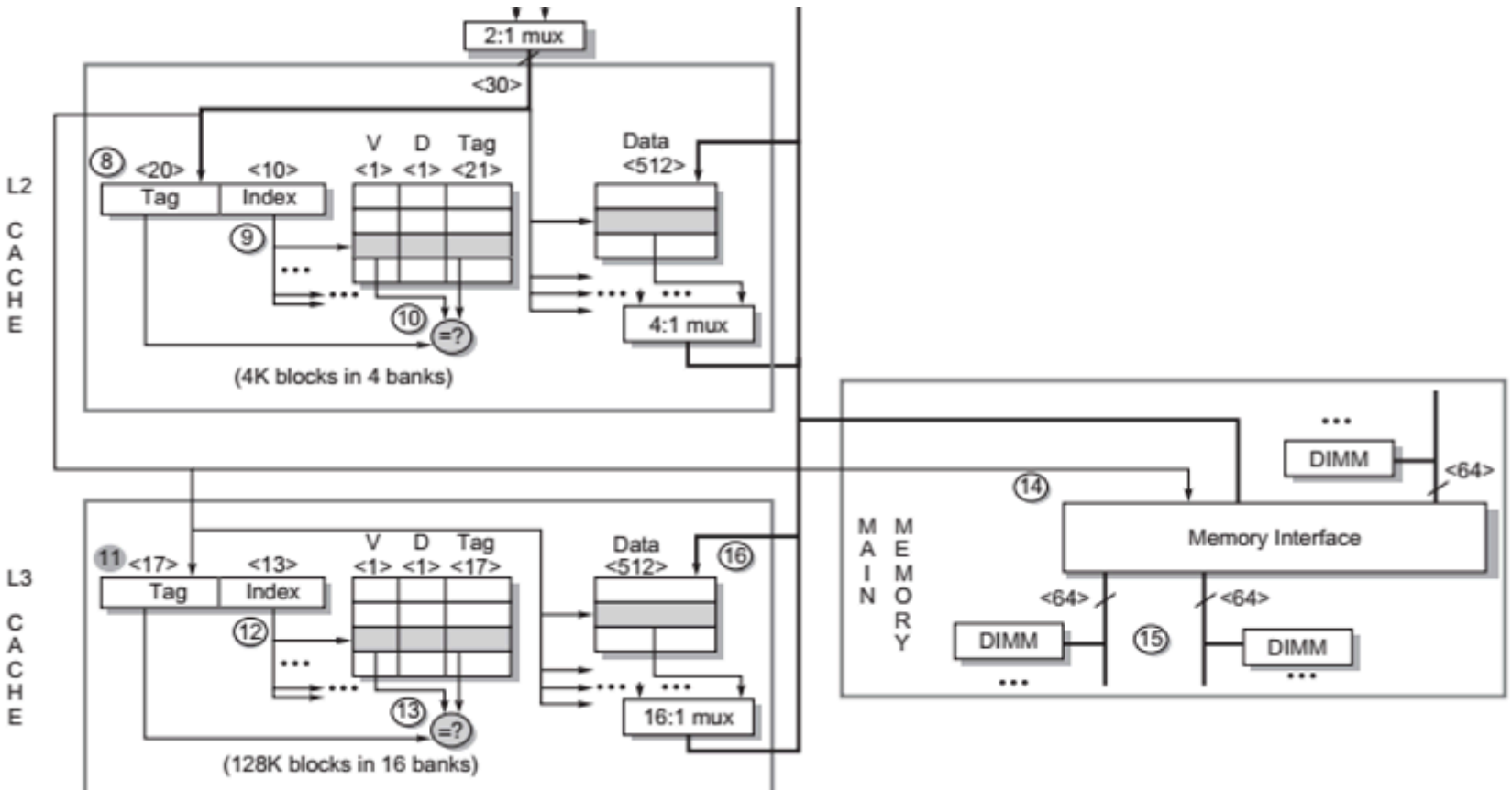
- количество наборов кэша в каждом slice кэша равно 2048
- количество блоков данных, имеющих один и тот же индекс набора кэша, равно

$$\frac{n_{block}}{n_{set}} = \frac{2^{28}}{2^{11}} = 2^{17}$$

Пример организации памяти в Intel Core i7 6700



Пример организации памяти в Intel Core i7 6700

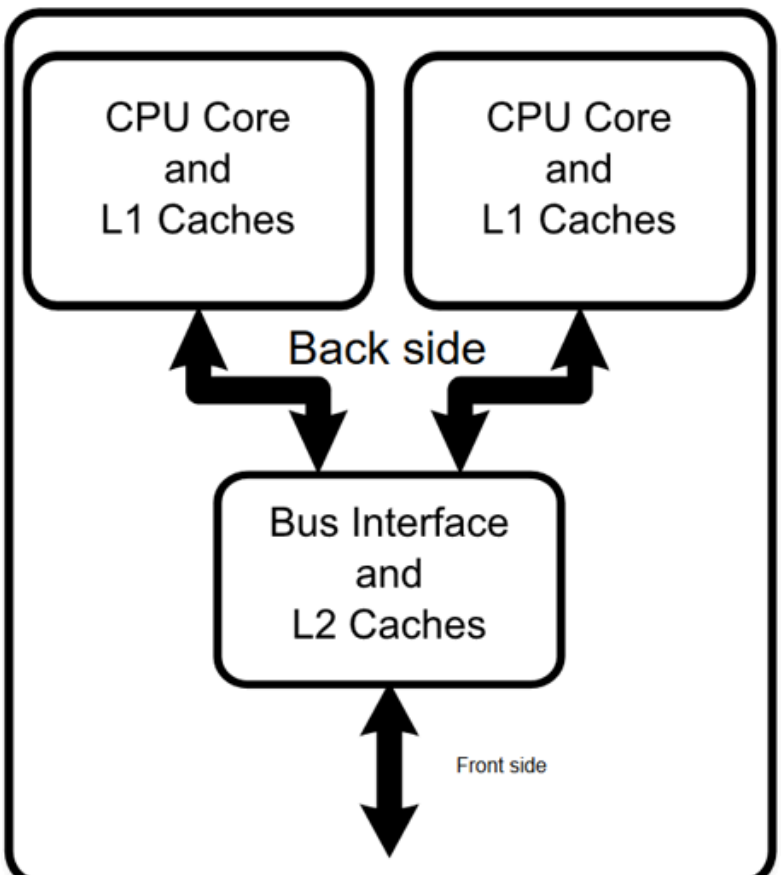


Дополнительная литература



- Физическая организация кэш-памяти процессора
- Экспериментальное определение характеристик кэш-памяти
- Аллергия эффектов кэшей процессоров
- Секреты кэш-памяти, или как потратить 1000 тактов на 10 команд

Проблема когерентности кэша



Варианты решения



- программист
  - барьеры памяти и методы синхронизации
- компилятор
  - барьеры памяти и методы синхронизации
- аппаратная реализация

Различие между многоядерными и многопроцессорными реализациями



- техника «подслушивания» (snooping)
  - общая шина
    - считается надежной средой
    - клиент не ожидает окончания рассылки
    - условие: требуется сообщить об ошибке
    - broadcast-рассылка

Cache-coherency protocol



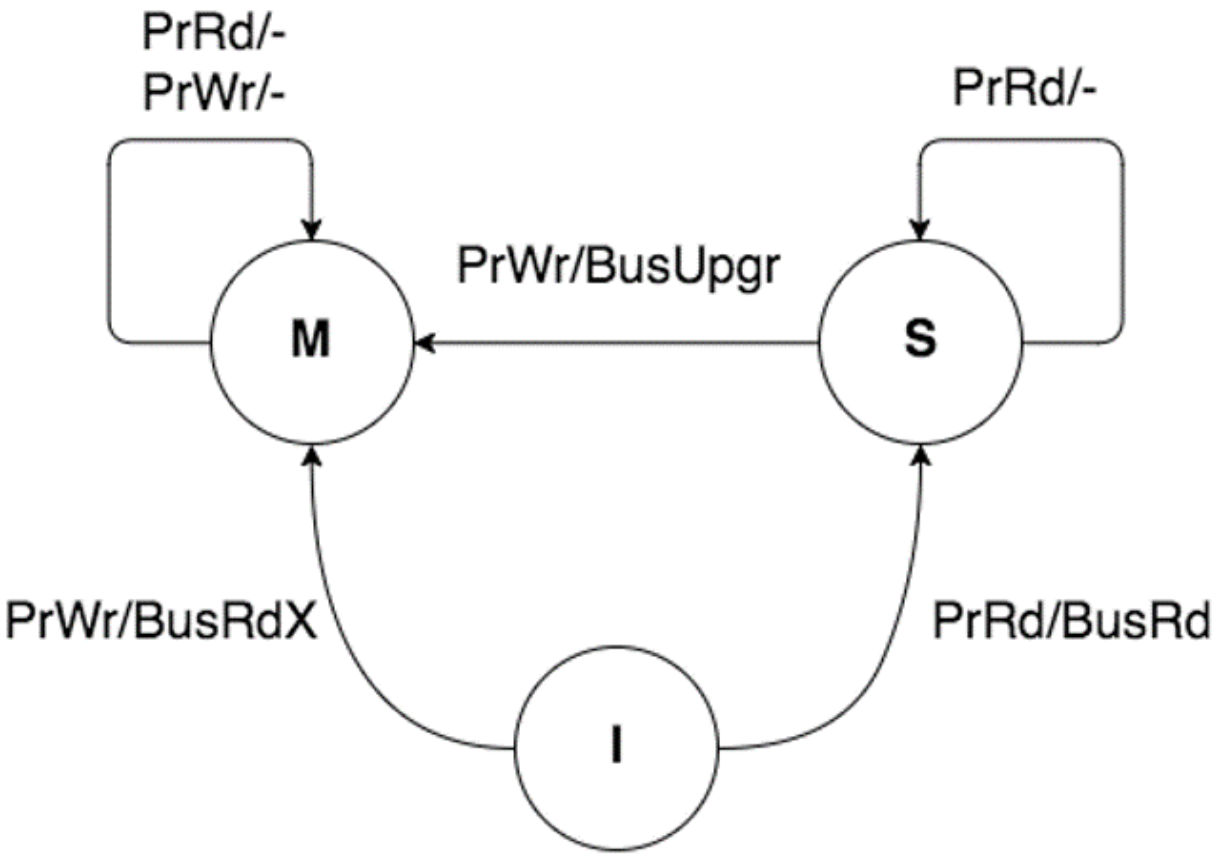
- MSI
- MESI (Intel Pentium, Intel Core)
- MOSI
- MOESI (AMD Opteron)
- MOWESI
- MERSI
- MESIF (Intel Nehalem)
- Firefly (DEC)
- Dragon (Xerox)
- …

Протокол MSI

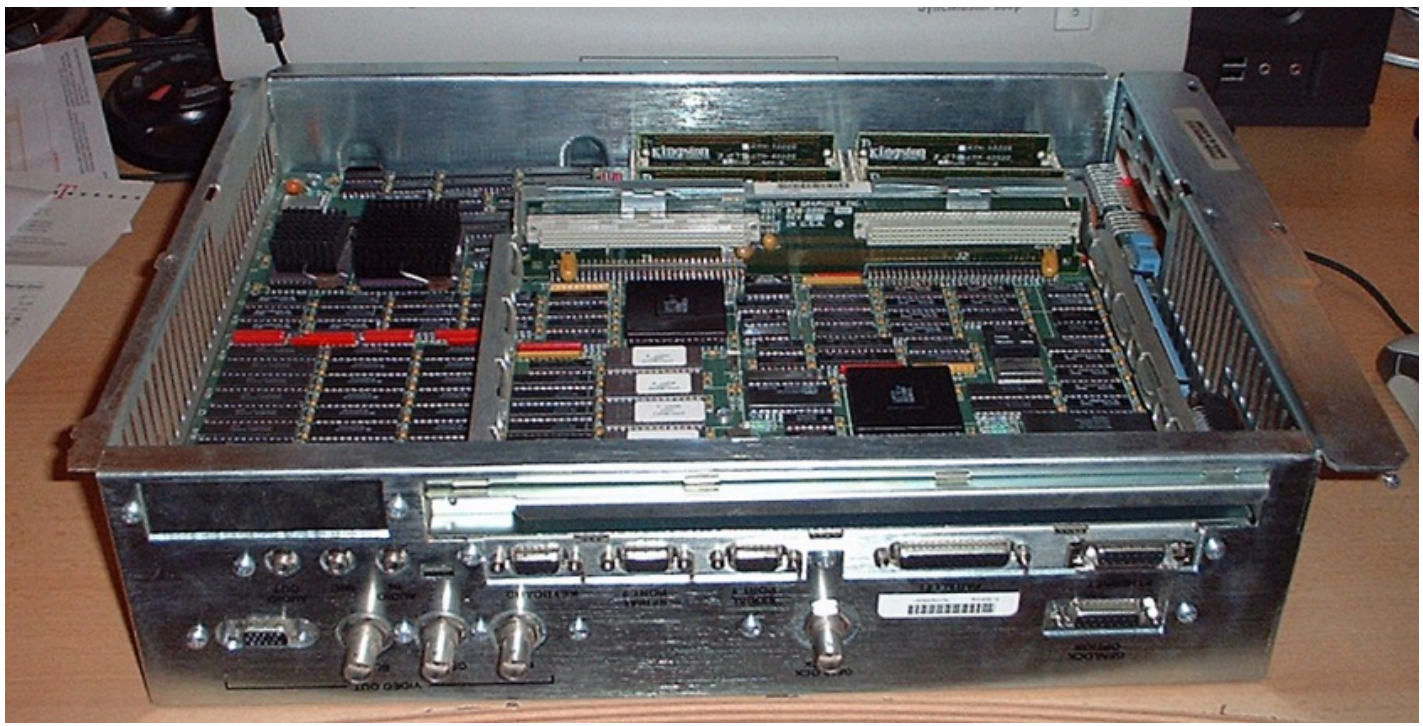


- 3 состояния:
  - Modified: блок изменен в кэше и не соответствует в O. Только один кэш в текущий момент может быть в данном состоянии
  - Shared: блок не изменен и существует как минимум в одном из кэшей в режиме только чтения
  - Invalid: блок не существует

Протокол MSI



Применение



SGI 4D machine (ближний аналог)

Протокол MSI



- взаимодействие всегда через ОЗУ
- состояние M:
  - сброс в память только при необходимости
- состояние S:
  - при переходе в M - оповещение

Протокол MESI



- 4 состояния:
  - Invalid: значения нет в кэше
  - Exclusive: значение есть только в этом кэше, и оно пока не было изменено
  - Modified: значение изменено этим процессором, и оно пока не находится ни в главной памяти, ни в кэше какого-либо другого процессора
  - Shared: значение присутствует в кэше более чем у одного процессора (не строгое состояние)
- переходы:
  - Отправка сообщения

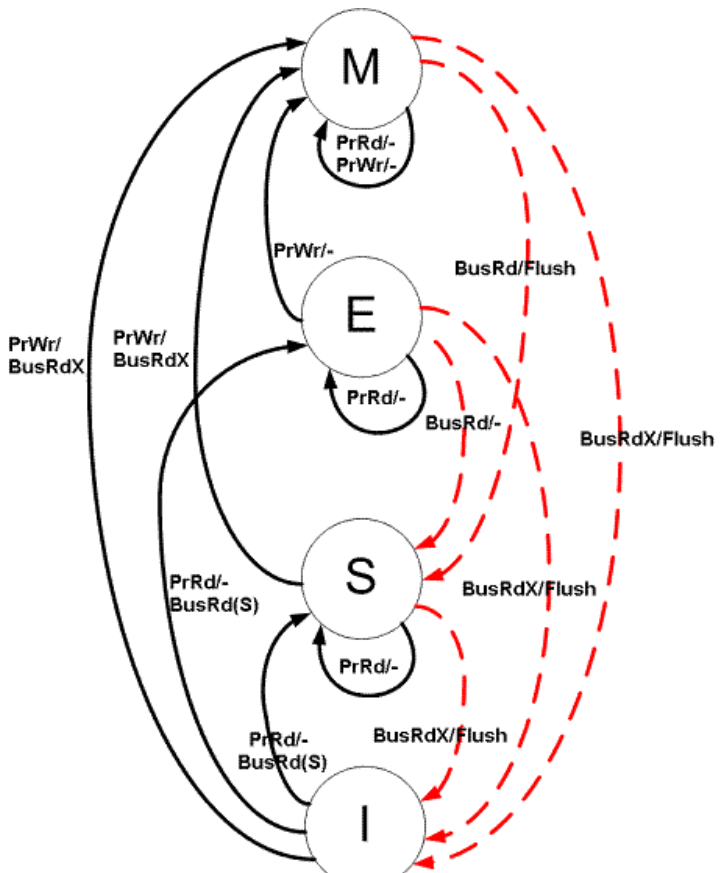
Протокол MESI



- Mark S. Papamarcos, Janak H. Patel. A Low-Overhead Coherence Solution for Multiprocessors with Private Cache Memories (Proceedings of the 11th Annual International Symposium on Computer Architecture, ISCA'84)

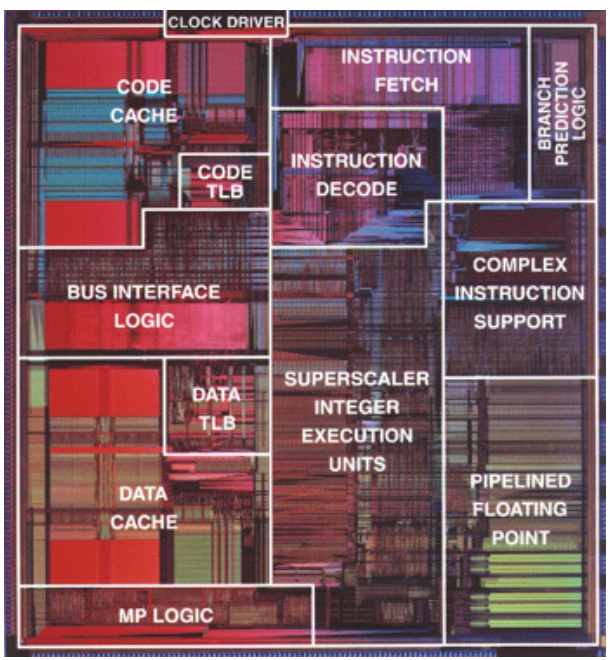
Диаграмма переходов из состояния в состояние

- Красная стрелка: инициирует транзакцию шина
- Черная стрелка: инициирует транзакцию процессор



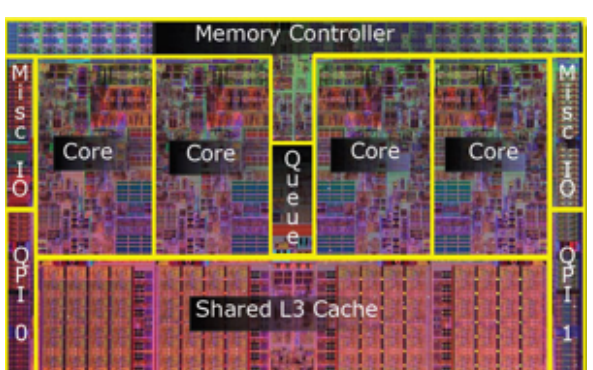
Преимущества MESI

- Наличие состояния Exclusive
  - Нет необходимости проверять актуальность данных в других ядрах/процессорах
- Применение:
  - Intel Pentium
  - AMD K6
  - PowerPC 601



Протокол MESIF

- Идея:
  - Попытка получить данные от других уровней кэша
- Проблема:
  - Состояние S - сообщения приходят от всех
- Решение:
  - F(orwarded) Разновидность состояния S «первый среди равных»
    - Состояние получает всегда последний записавший данные
- Применение:
  - Intel Nehalem

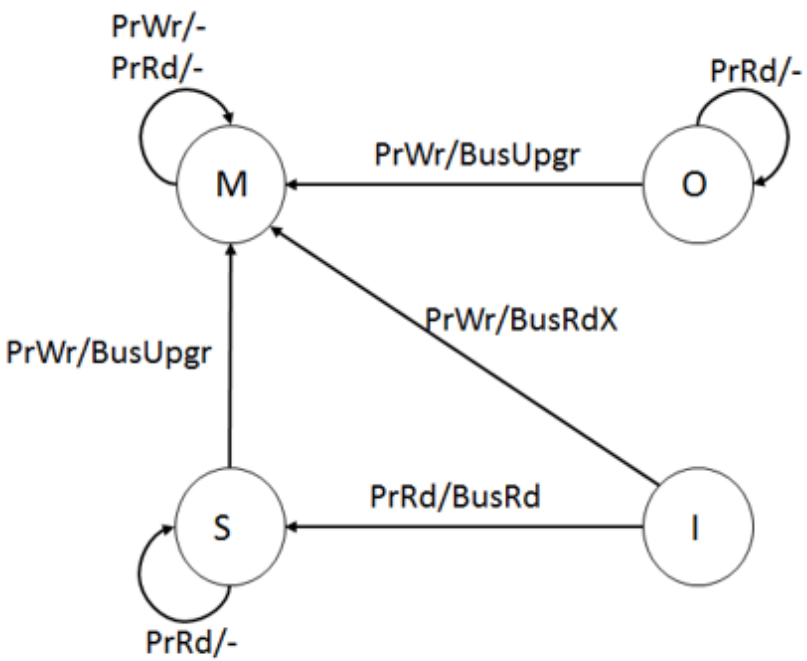


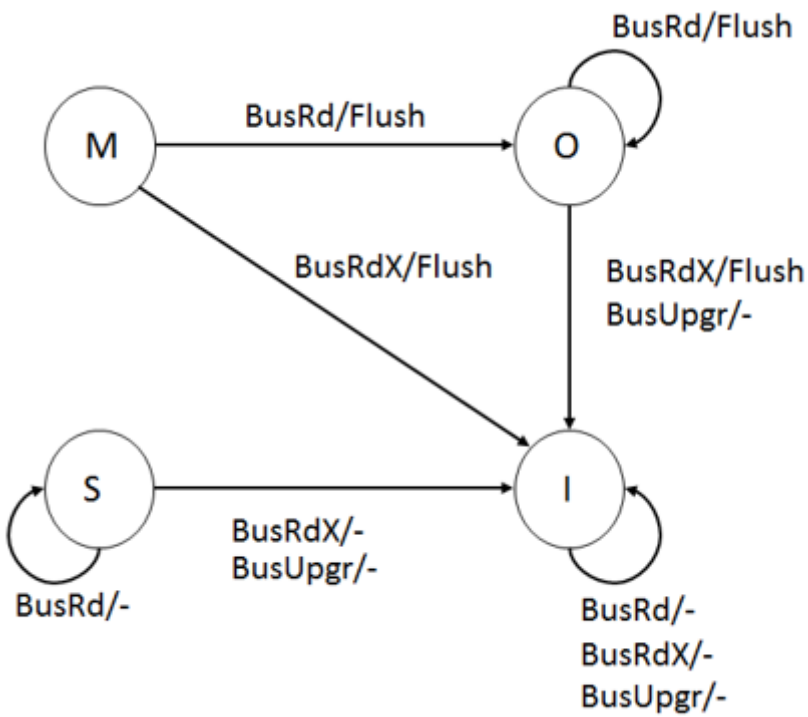
Протокол MOSI



- Проблема MSI (данные в модифицированном состоянии):
  - Отмена транзакции -> сброс данных в основную память -> смена состояния на Invalidate -> перезапрос копии
- Решение:
  - O(wned) Состояние указывает, что данный кэш владеет эксклюзивными данными (отличие от MSI: в память данные не актуальны !!!)

Протокол MOSI





Отличия MSI / MESI / MOSI



- MSI / MOSI:
  - Состояние O - оптимизация сброса данных в память (write-back)
- MSI / MESI:
  - Состояние E - оптимизация числа транзакций, необходимых для чтения/записи от другого процессора (ядро)

Протокол MOESI



- Modified: блок изменен в кэше и не соответствует в O. Только один кэш в текущий момент может быть в данном состоянии
- O(wned) Состояние указывает, что данный кэш владеет эксклюзивными данными
- E(xclusive)
- Shared: блок не изменен и существует как минимум в одном из кэшей в режиме только чтения
- Invalid: блок не существует

Протокол MOESI



- AMD64 architecture
- SUN Microsystems UltraSPARC

Дополнительная литература



- Мифы о кэше процессора, в которые верят программисты

РЕГИСТРОВЫЙ ФАЙЛ

Разрядность архитектуры

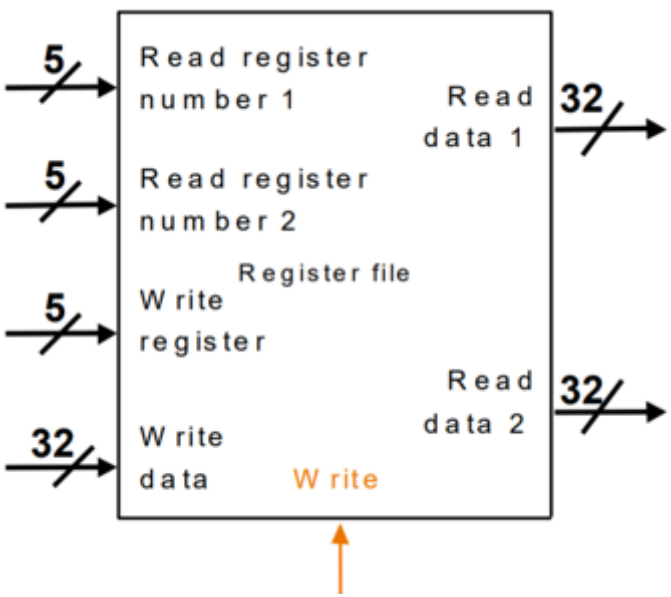


- Разрядность виртуального адреса
- Максимальная разрядность целочисленного операнда
- Максимальная разрядность универсального регистра

Регистровый файл (register file)



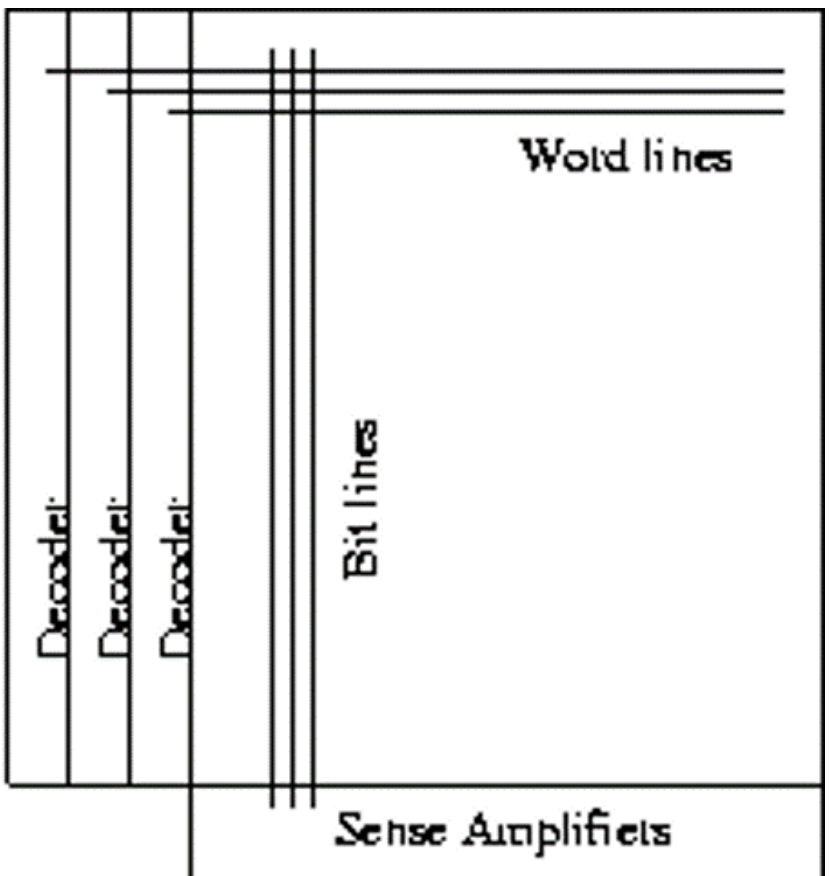
- Модуль микропроцессора (CPU), содержащий в себе реализацию регистров процессора
  - т.е. физическая реализация регистров, описанных в ISA
- Реализован как многопортовая SRAM (описание)



Пример регистрового файла в MIPS архитектура

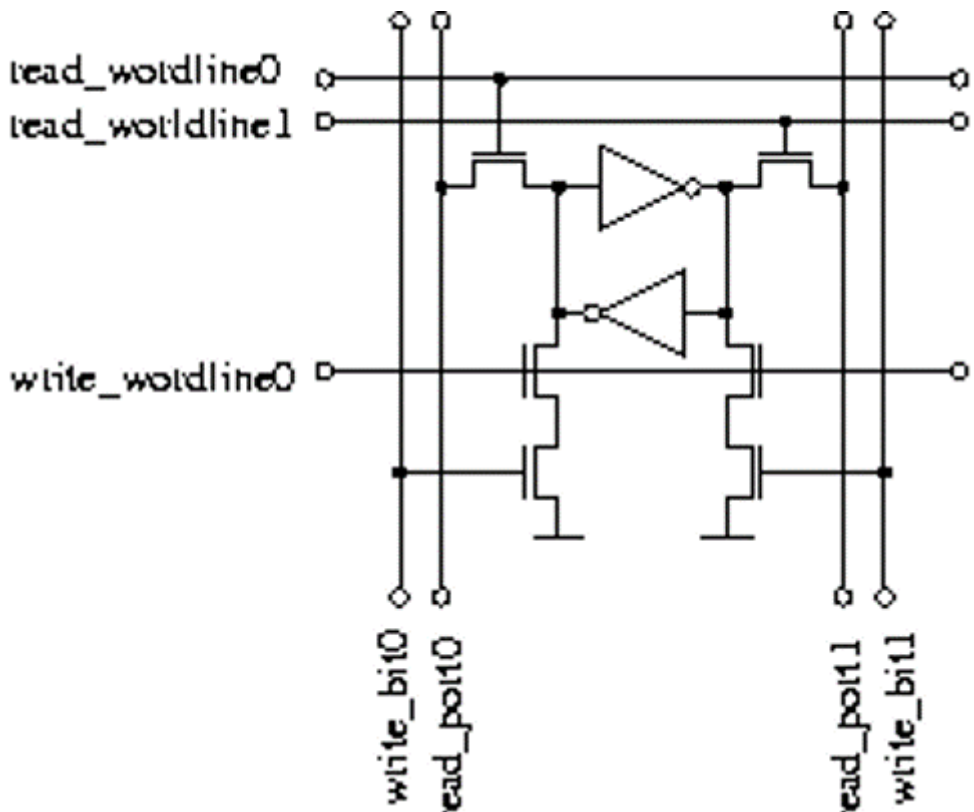
Внутренняя реализация

- Массив запоминающих ячеек SRAM подключается к Word Lines
- С помощью Decoder данные выставляются на шину Bit lines



Пример статической памяти с 3 портами ввода/вывода

- Типичный регистровый файл с 3 портами для параллельного доступа:
  - Чтение из 2 регистров
  - Запись в 1 регистр



Зачем это надо



- Внеочередное исполнение (англ. out-of-order execution)
  - Исполнение машинных инструкций не в порядке следования в машинном коде, а **в порядке готовности** к выполнению
  - Основано на алгоритме Томасуло и его развитии
- Переименование регистров (англ. register renaming)
  - Способ реализации параллелизма на уровне команд

Переименование регистров



Версия 1	Версия 2

`r1 = m[1024]`
`r1 += 2`
`m[1032] = r1`
`r1 = m[2048]`
`r1 += 4`
`m[2056] = r1`
	`r1 = m[1024]`
	`r1 += 2`
	`m[1032] = r1`
	`r2 = m[2048]`
	`r2 += 4`
	`m[2056] = r2`

В чём разница между версиями 1 и 2???

Переименование регистров



- Переименование регистров основано на таком же подходе
  - между архитектурными регистрами (x86: eax, ebx, &) и физическими нет строгого соответствия
    - при этом физических регистров существенно больше
  - при исполнении на CPU архитектурному регистру назначается не постоянная физическая копия. Если блок исполнения фиксирует, что архитектурный регистр может быть заменен на другой физический без нарушения результата исполнения, данная замена осуществляется (!)
    - более подробно аппаратная часть будет в дальнейшем (алгоритм Томасуло)

Регистровый файл



- Классический
- Стековый
- С изменяемой разрядностью регистров
- Статический оконный
- Динамический оконный (регистровый стек)

Классический регистровый файл

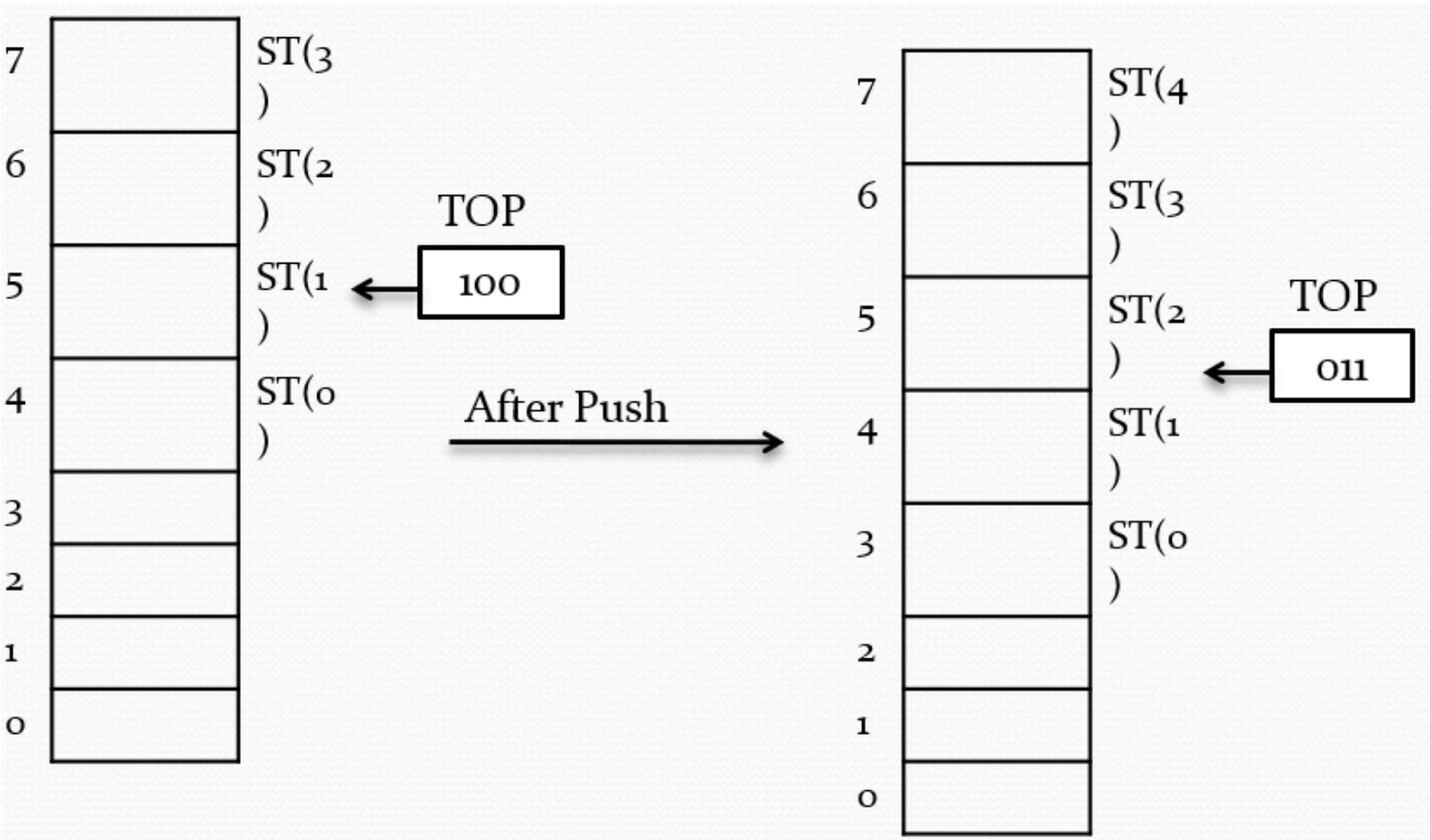


- Количество регистров: 8 - 32
- Разрядность регистров: 32 или 64 бита
- Достоинства
  - Простой
- Недостатки
  - Программное сохранение требуемых регистров при вызове процедур

Стековый регистровый файл



- Используется в архитектуре x87
- Позволяет неявно обращаться к операндам на вершине стека


7
6
5
4
3
2
1
0
ST(3)
ST(2)
ST(1)
ST(0)
TOP
100
After Push
7
6
5
4
3
2
1
0
ST(4)
ST(3)
ST(2)
ST(1)
ST(0)
TOP
011

Регистровый файл с изменяемой разрядностью регистров



- Используется в архитектуре вещественных сопроцессоров SPARC и PA-RISC

0	[31:0]
1	[31:0]
2	[31:0]
3	[31:0]
4	[31:0]
5	[31:0]
6	[31:0]
7	[31:0]

0	[31:0]
	[63:32]
1	[31:0]
	[63:32]
2	[31:0]
	[63:32]
3	[31:0]
	[63:32]

0	[31:0]
	[63:32]
	[95:64]
	[127:96]
1	[31:0]
	[63:32]
	[95:64]
	[127:96]

Проблемы и недостатки предыдущих подходов при вызове процедур



- Сохранение регистров
  - Вообще не сохранять
  - Сохранение в память
- Передача параметров
  - Через регистры
  - Через память
- Возврат значений

Оконный регистровый файл



- Используется в архитектуре SPARC
- Большое количество регистров: до 640
  - По материалам Oracle SPARC Architecture 2015
    - Есть ли новое???
  - Доступ к регистрам осуществляется через окно фиксированного размера: 32 регистра
  - Окно разбито на четыре части
    - Глобальные регистры
    - Локальные регистры
    - Собственные входные/выходные параметры
    - Входные/выходные параметры вызываемой функции
  - Регистр CWP (Current Window Pointer register)
  - Альтернативные глобальные регистры

Схема оконного регистрового файла

- Для организации одного локального окна требуется 16 регистров
- Глобальные регистры при переключении между окнами не изменяются
- При переполнении регистрового файла генерируется исключение, по которому ОС может очистить несколько регистровых окон

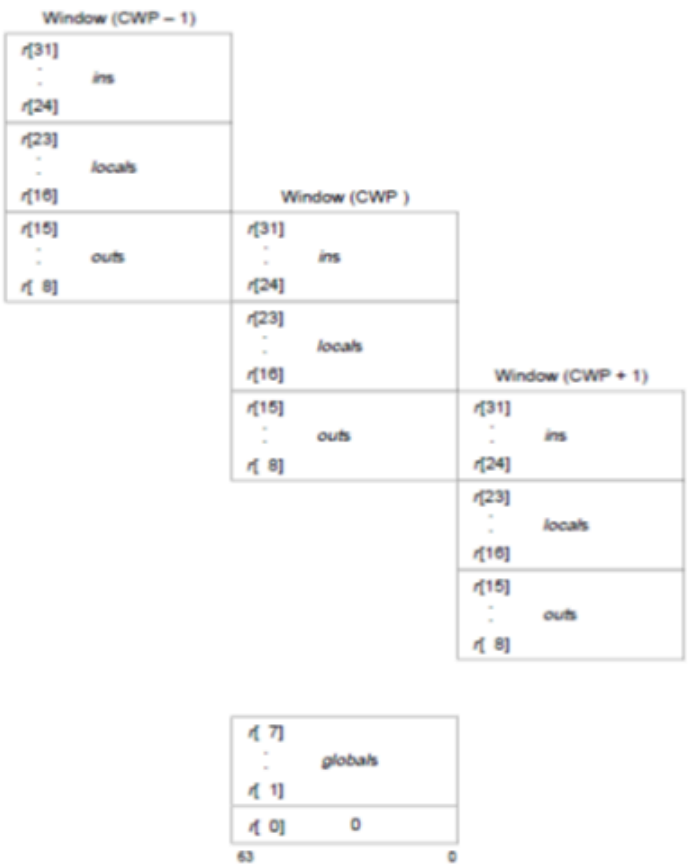
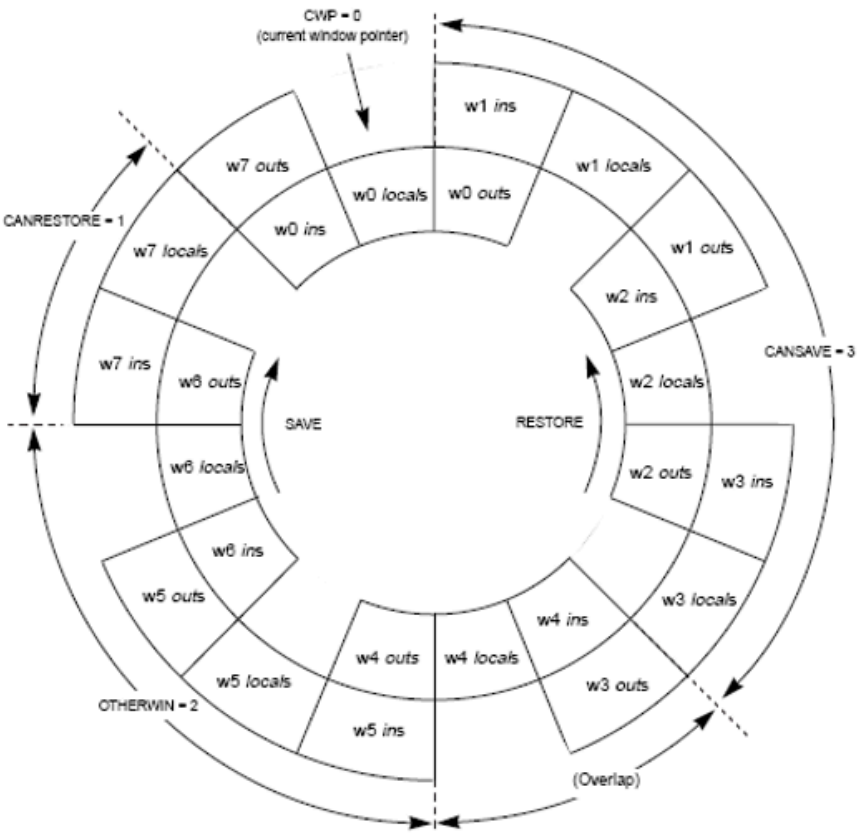


Схема перекрытия окон



Достоинства и недостатки



- Достоинства:
  - Передача параметров между процедурами происходит через регистры
  - Можно избежать необходимости сохранять регистры в память
- Недостатки:
  - Низкая скорость регистрового файла
  - Генерация исключения при переполнении регистрового файла
  - Недостаточная гибкость:
    - Фиксированный размер окна
    - Фиксированное количество локальных регистров и регистров, которые используются для передачи параметров

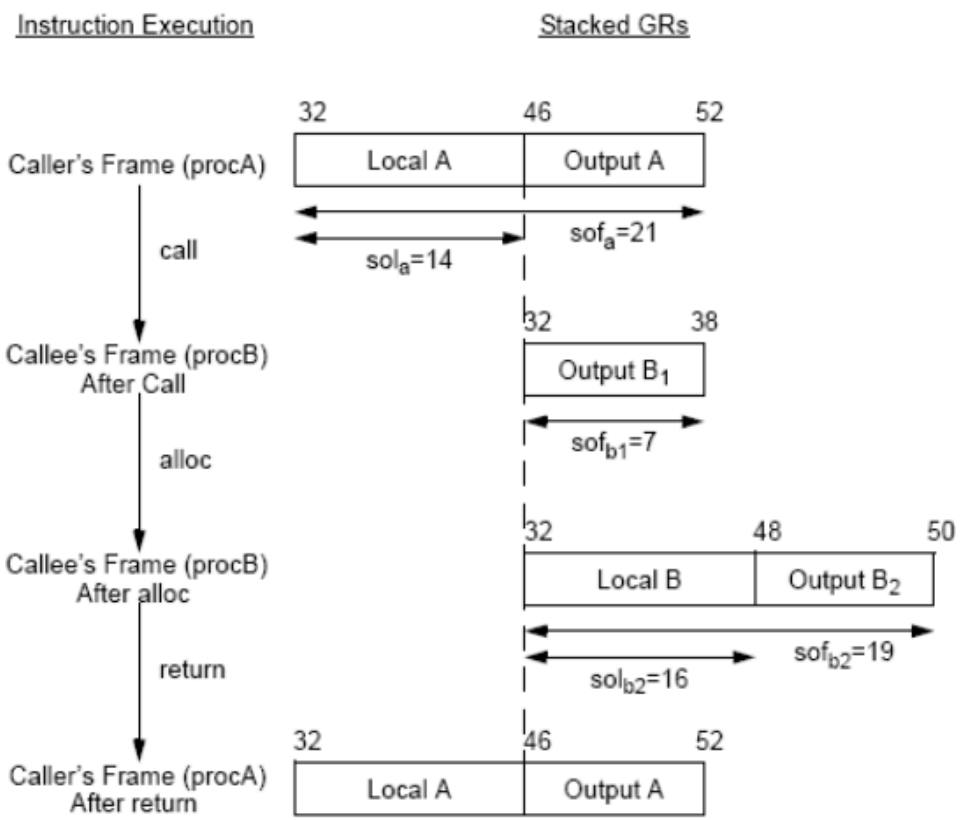
Динамический оконный регистровый файл или регистровый стек



- Используется в архитектуре IA-64
- Регистровое окно произвольного размера
- Количество локальных регистров и регистров входных/выходных параметров произвольно
- Использование механизма Register Stack Engine (RSE) для автоматического сохранения/загрузки регистров в случае переполнения регистрового файла
- Количество глобальных регистров: 32, стековых регистров 96.

Схема функционирования регистрового стека

- Специальная инструкция:
  - alloc sof, sol
  - sof - size of frame
  - sol - size of locale



Достоинства и недостатки



- Достоинства:
  - Не нужно заботиться о сохранении/загрузке регистров при вызове процедур
  - Эффективное использование регистров из-за изменяемого размера окна
- Недостатки:
  - Высокая сложность аппаратной реализации RSE

Дополнительная литература



- Oracle SPARC Architecture 2015
- Understanding stacks and registers in the SPARC architecture